

第二课：第一个 Agent 应用实战——从“理论”到“动手”的跨越

01

课程主题

项目	内容
课程主题	第一个 Agent 应用实战
课时安排	第1周第2次课（2小时）
学员基础	已掌握 Agent 基本概念，完成 OpenClaw 环境搭建
教学目标	1. 掌握 Agent 应用开发框架与架构设计思路 2. 基于 LangChain 完成第一个Agent实战 3. 理解并实现 Tools+Memory 的集成 4. 通过 OpenClaw 快速上手对比，建立框架感知

02

环境准备

- Python 3.11+ 环境
- 大模型API密钥（余额充足）
- 上节课已完成的 OpenClaw 环境
- 提前安装依赖包：`pip install langchain langchain-openai python-dotenv`

03

课程详情

第一部分：开场与上节回顾

这是 AI Agent 全栈开发课程的第二课。

上节课，我们完成了一场智能体 ‘认知革命’ ——从 Agent 的定义到四要素公式。

从技术演进路线再到 OpenClaw 环境搭建，我们尝试把 Agent 从 ‘黑盒子’ 变成了 ‘透明箱子’ 。

也许大家心里有一个疑问：理论听懂了，但是作为技术人员，我们代码应该怎么写？

Agent 的这些组件——大脑、规划、记忆、工具——怎么通过代码让它们真正跑起来，从而帮我们干活和解决问题？

做技术不能眼高手低，接下来我们就来解决这个问题。

1.1 【上节课核心回顾】

“在动手之前，我们还是先快速回顾一下上节课的几个关键点，确保大家基础是牢固的：

第一：Agent 公式

代码块

```
1 Agent = LLM（大脑） + Planning（规划） + Memory（记忆） + Tool Use（工具调用）
```

这四大组件缺一不可。它们集体构成了智能体的基本骨架。

第二：技术演进四阶段

代码块

```
1 从基础API调用 → 提示词工程 → Function Calling → Agent框架
```

特别是 Function Calling，它让模型从 ‘说话’ 进化到 ‘发指令’ ，是 Agent 能够 ‘动手’ 的技术前提。

第三：OpenClaw是什么？

代码块

上节课结束时，我们还搭建了 OpenClaw 环境，但还没有用它实现具体的业务。

今天，我们写代码实现一个完整的 Agent，然后再用 OpenClaw 实现同样的功能——通过对比，建立对不同框架的直观感受。

1.2 【本节课目标与结构】

本节课我们的目标就是：实战第一个 Agent。

我们将实现三个目标，而且这三个目标层层递进：

第一个目标：理解 Agent 应用的架构设计

在写代码之前，先建立工程思维。我们介绍经典的三层架构——表现层、业务层、能力层。

这就像盖房子需要先画图纸，避免‘拿到需求直接写代码’的混乱。

第二个目标：动手写两个实战项目

- 实战1：开发基础 Agent —— 基于 LangChain 框架，实现一个对话 Agent，不加任何工具，起到快速上手目的。
- 实战2：引入 Tool+Memory——给 Agent 添加‘获取当前时间’和‘计算器’两个工具，并实现短期记忆，从而让 Agent 不仅能干活，还能记住你是谁。

第三个目标：OpenClaw 快速上手并与 LangChain 框架进行对比

我们会使用 OpenClaw 实现同样功能，并体验 OpenClaw 的‘技能（Skill）’机制，然后我们对比 LangChain 和 OpenClaw 的设计哲学以及它们各自的适用场景。

最后老规矩，我们会简单总结本节课的核心知识点，然后布置课后作业，并预告下节课的内容。

1.3 【为什么先讲 LangChain 再讲 OpenClaw？】

当然，可能有同学会问：既然 OpenClaw 这么火，为什么不直接学 OpenClaw，还要先讲 LangChain？

原因很简单，主要有以下两个：

1. Openclaw 最近才出现，而 langchain 已经存在几年了。企业以前用 langchain 开发好的项目如果运行正常，基于性价比，不会轻易将项目迁移到 openclaw。你如果入职这样的企业，需要有维护 langchain 项目的能力。
2. LangChain 是理解 Agent 底层机制的‘教科书’。OpenClaw 封装了很多细节，用起来确实方便，但如果你不明白背后发生了什么，遇到复杂问题时就很难调试。LangChain 的设计更贴近底层，你需要显式地定义工具、配置记忆、构建执行图——整个过程是一次深度的学习，对于你后期的进化按下了加速键。

打个比方：学开车，你可以直接开自动挡，很快就能上路；但如果你想成为真正的赛车手，就必须学好手动挡，理解离合器、档位、油门之间的配合。LangChain 就是 Agent 世界的‘手动挡’，而 OpenClaw 则是‘自动挡’。

因此，在今天的课程里，我们会先用 LangChain 把 Agent 每个零件亲手组装起来，让你看到它内部是怎么运转的；然后再用 OpenClaw 实现同样的功能，让你感受‘自动挡’的便捷。这样你既能打好基础，又能跟上行业主流。

1.4 【过渡：进入架构设计】

目标已经清楚了。接下来，我们先迈出第一步：理解 Agent 架构设计。

在写第一行代码之前，我们先回答自己一个问题：一个生产级的 Agent 应用，它的代码应该怎么组织？才能保证以后好维护、好扩展？

这就要引入 Agent 的三层架构设计了。

接下来我会带大家画出这张架构图，并解释每一层的作用。

第二部分：Agent应用开发框架——从架构设计到工程实践

2.1 为什么要先谈架构？

2.1.1 【问题引入：新手常犯的错误】

很多同学第一次写 Agent，容易犯一个错误：拿到需求直接写代码，想到哪写到哪。

即使是工作很多年的老程序员，在 Vibe coding 时代，也会变得无所适从，因为怕老板嫌弃进度慢，从而放弃经典软件编码时代被培养起来的工程化习惯。

当领导说‘帮我快速实现个智能客服’，我们可能会立刻打开编辑器，噼里啪啦开始写，或者直接使用 AI 编码器开始 Vibe coding，

- 先调大模型 API，把程序跑通再说
- 客户说我要查天气，然后你加个天气预报API的功能集成
- 过两天客户说‘需要能查订单’，然后你又加了个if判断
- 再过一周客户又‘需要它能记住用户偏好’，你又找了个地方塞了个字典等等……

结果三个月后，代码长成了这样：

代码块

```
1  def my_agent(user_input):
2      # 这里是查天气的逻辑
3      if '天气' in user_input:
4          # 调用天气API
5          ...
6      # 这里是查订单的逻辑
7      elif '订单' in user_input:
8          # 调用订单API
9          ...
10     # 这里是... 1000行代码后
```

然后逻辑乱了，工具调用不灵了，记忆不知道放哪了，加一个新功能要改十几个地方。

这个过程就像盖房子不打地基，直接砌墙——也许前两堵墙能立住，但盖到三楼的时候，你发现墙歪了、承重不够、水电管线没预留……最后只能推倒重来。

所以，这里给大家提个醒：在写第一行代码之前，我们要先搞清楚：一个生产级的 Agent 应用，应该怎么组织它的代码结构？

2.1.2 【架构的价值：不是束缚，是保障】

有的同学也许会反驳：‘我只是写个小Demo而已，至于这么讲究吗？’

这种求速的心理我能理解，但往往‘小Demo’和‘生产级应用’之间，往往只需要一个客户的需求变更。

当你写第一个 Agent 时，它可能只有两个工具。但第二周，需求变成了五个工具。第三周，要求跨会话记忆。第四周，要接入飞书机器人。如果我们没有清晰的架构，每一次扩展都是一次重构。

架构不是束缚，是保障。它会帮你做三件事：

1. **隔离变化**：换个模型，只改一个文件；加个新工具，只动一个目录。

2. **降低认知负担：**当你三个月后再看这段代码，你能立刻知道‘模型配置在哪里’、‘工具定义在哪里’、‘业务逻辑在哪里’。
3. **便于协作：**如果团队里有多个人开发，那么架构可以让我们每个人都知道自己的代码该放哪里，从而避免冲突。

所以，今天我不只是带大家怎么写 Agent，也是建议大家怎么正确组织智能体 Agent 架构。

2.1.3 【引入三层架构】

那么，好的架构长什么样？

经过大量项目实践，业界总结出一个经典模式——那就是三层架构。

这三层分别是：

- **能力层：**最底层，封装所有可复用的能力——比如：模型适配器、工具函数、存储记忆等等。这一层只关心‘能做什么’，不关心‘什么时候做’。
- **业务层：**中间层，负责Agent的决策逻辑——比如：意图识别、任务规划、结果整合。这一层是Agent的‘大脑’，决定‘怎么做’。
- **表现层：**最上层，负责与用户交互——可以是命令行、Web API、飞书机器人。这一层只负责‘输入输出’，不包含任何业务逻辑。

接下来，我会带大家逐层拆解，看看每一层到底放什么代码，以及它们之间怎么配合。

2.2 Agent应用的三层架构

2.2.2 【架构图】

我们先来画 Agent 三层架构‘图纸’。这张图从上到下，分别是表现层、业务层、能力层。

每一层都有明确的职责，就像一家公司：前台表现层负责接待客户，中层业务层负责决策和执行，后台能力层负责提供资源。

2.2.3 【第一层：能力层——Agent的“工具箱”】

先看最底层：能力层。

这一层放的是什么呢？答案：是 Agent 所有可复用的基础能力。

这一层不关心用户说了什么，也不关心现在该不该调用——它只负责一件事：我有什么本事。

能力层包含三个核心模块：

第一个：**模型适配器**。

因为不同的大模型可能它们各自的 API 格式不同——比如 OpenAI 有自己的风格，智谱也有自己的一套风格，月之暗面又是一套。模型适配器的作用就是统一各个模型的 API 调用接口：它不管你底层是 GPT-4 还是 Qwen，对业务层提供统一的调用接口。这样设计如果你想换模型时，只改一行配置，业务层可以做到完全无感知。

第二个：**工具集**。

工具集是 Agent 的‘手脚’。查天气、计算器、发邮件、查数据库等等所有能调用的函数都放在这里。每个工具也应该设计成都是独立的，这样可以随时增加或替换。

第三个：**记忆模块**。

负责管理会话短期记忆以及向量数据库的长期记忆。它只管怎么存、怎么取，它不管什么时候取——因为那是业务层该做的事。

我们对能力层的总结就是：可复用、可插拔。就像一个工具箱，锤子、螺丝刀、电钻都放好了，用的时候随时拿，有更高级的工具出现时就随便更新。

2.2.4 【第二层：业务层——Agent的“大脑”】

再往上一层：业务层。

这是Agent的决策中枢，也是整个架构的核心。如果说能力层是‘能做什么’，业务层就是‘怎么做’。

业务层通常也包含三个子模块：

第一个：**意图识别**。

当用户说‘帮我策划明天的出差行程’，业务层要理解这不是闲聊，而是一个需要执行的任务。它会分析用户的输入，判断属于哪类意图——是简单问答，还是复杂任务。

第二个：**任务规划**。

这个模块是业务层的‘导演’。当识别用户发出的是复杂任务后，规划模块会把用户复杂任务拆解成可以执行的步骤。比如‘策划出差’这个任务就会被拆成：查天气→查机票→订酒店→创建日历→约车。这个拆解是动态实施的，而不是写死的。

第三个：**结果整合**。

当上面任务的各个步骤都执行完，可能会返回零散的信息——比如天气数据、航班信息、酒店确认号。这时就需要整合模块把这些碎片信息拼成一份完整的、对用户友好的出行建议返回给最终用户。

业务层的工作方式其实就是我们上节课介绍的 ReAct 循环：思考环节就是这里的意图识别+规划、行动就是能力层的调用、观察就是获取碎片化的整合结果、然后再进入思考、行动、观察循环，直到最终任务完成。

我们对业务层的总结就是：**决策、编排**。它像一个经验丰富的项目经理，知道什么时候该用什么工具，怎么把零散结果串成最终交付物。

2.2.5【第三层：表现层——Agent的“面孔”】

最上面一层是表现层。

这是用户直接接触的部分，也是Agent的‘面孔’。表现层负责两件事：接收输入、展示输出。

它的形式可以有多种，比如：

- 最简单的：命令行界面（CLI），适合开发调试。
- 更常见的：REST API，供其他系统调用。
- 也可以是聊天平台：飞书、微信、Slack、Discord 等等

有关表现层的关键设计原则是：这一层要做得尽可能薄。

什么意思？表现层只做输入输出的‘翻译’工作，绝对不要包含任何的业务逻辑。比如：

- 收到飞书消息，提取出文本内容，传给业务层
- 业务层返回结果，包装成飞书消息格式，发回去

为什么必须薄？因为如果你把业务逻辑写在表现层里，以后想从飞书切到微信，就得把逻辑重写一遍——这就非常糟糕了。

我们对表现层的总结就是：薄、可替换。就像手机壳，换壳不影响手机内部零件。

2.2.6【总结：关注点分离的价值】

好，三层都讲完了。现在我们来总结一下这个架构的核心思想——五个字：**关注点分离**。

- 能力层只管‘能做什么’——我有哪些工具、支持什么模型、怎么存记忆。
- 业务层只管‘怎么决策’——用户想干什么、怎么拆解、怎么整合。
- 表现层只管‘怎么交互’——用什么渠道接收输入、什么渠道展示输出。

经过这样的分层后，最大的好处就是：你可以轻松替换任何一层，而不用重写整个 Agent。

举个例子：

- 你想把模型从GPT-4换成Claude？只改能力层的模型适配器，业务层和表现层纹丝不动。

- 你想增加一个新工具（比如查汇率）？只在能力层加一个函数，然后在业务层的提示词里提一句，完事。
- 你想把前端从命令行换成飞书机器人？只写一个新的表现层，能力层和业务层直接复用。

这就是架构的力量：让代码有生命力，而不是一改就散。

2.3 LangChain 在这三层中的角色

2.3.1 【LangChain是“脚手架”还是“预制板”？】

好，三层架构蓝图我们已经解释清楚了。现在问题来了：我们是不是需要从零开始，自己实现架构的每一层？

答案是：不用。如果每写一个 Agent 都要自己写 HTTP 客户端、自己实现 ReAct 循环、自己管理记忆存储，那开发效率也太低了。

这就是框架的作用。

今天我们要用的框架——LangChain。已经在这三层实现了非常重要的功能。

2.3.2 【能力层：LangChain帮我们“预制”了所有基础能力】

首先我们来看能力层。

LangChain 提供了大量现成的组件，让我们不用重复制作轮子：

- 模型适配器：`ChatOpenAI`、`ChatAnthropic`、`ChatZhipuAI` ……几十个模型的统一接口，你只需要换一个类名，就能切换模型。不用自己封装API签名、处理处理重试、实现解析响应等等。
- 工具集：LangChain 内置了几十个常用工具——搜索、计算器、Python 解释器、Shell 命令、数据库查询等等……你也可以用 `@tool` 装饰器把自己的函数封装成工具。
- 记忆模块：提供了多种记忆实现——`ConversationBufferMemory`（用来缓存对话）、`VectorStoreRetrieverMemory`（用来进行向量检索）、以及基于 LangGraph 的更强大的检查点机制。

所以，在能力层，LangChain 就像一套预制好的‘工具箱’：你不需要自己去敲锤子、造螺丝刀，直接拿来就能用。

举个例子，我们上节课提到的 `get_current_time` 工具，如果用 LangChain 写，只需要：

```
1 from langchain_core.tools import tool
2
3 @tool
4 def get_current_time():
5     return datetime.now().strftime("%Y-%m-%d %H:%M:%S")
```

就这么简单。@tool 装饰器自动把函数转换成 LangChain 可以识别的工具，模型通过函数的文档字符串介绍，从而理解该函数工具的作用，并在需要时调用。

2.3.3 【业务层：LangChain帮我们“编排”决策流程】

再看业务层。

我们已经知道了业务层是 Agent 的‘大脑’，负责意图识别、任务规划、结果整合。

如果这些都要我们自己实现，我们需要写 ReAct 循环、处理工具调用以及管理思考-行动-观察的状态……代码量不小，而且容易出错。

而 LangChain 在这里同样提供了强大的支持：

- create_agent：这个函数就能帮我们实现一个完整的 ReAct Agent。你只需要给这个 Agent 传入具体模型、工具列表、系统提示词，它内部会自动构建出思考-行动-观察的循环，并处理工具调用。
- AgentExecutor：它负责执行 Agent，管理迭代次数、错误重试、中止条件等。
- LangGraph：对于更复杂的流程（多步规划、分支、循环），LangGraph 提供了图结构的工作流编排能力，让你可以用节点和边来描述业务逻辑，比硬编码循环更灵活。

所以，在业务层，LangChain 相当于一个‘决策引擎’：你只需要通过指定具体模型来定义好‘大脑’、通过选择合适的工具来做‘手脚’，同时借助提示词来定义‘行为准则’，剩下的循环、调度、状态管理等脏活累活，LangChain 框架可以帮你轻松搞定。

这也是为什么我们可以用不到20行代码，就可以写出一个能干活、且具有记忆的智能体 Agent。

2.3.4 【表现层：LangChain“放手”让我们自由发挥】

最后我们看表现层。

表现层负责与用户交互——比如通过：命令行、Web API、聊天平台等等。而 LangChain 在这方面不做任何预设。

它没有内置的命令行界面，也没有飞书机器人插件。为什么？因为表现层和业务逻辑是正交的——你可以用任何你喜欢的方式接入 Agent。

- 想快速测试？就写几行 `input()` 和 `print()`，这就是一个命令行 Agent。
- 想提供API？用 FastAPI 或 Flask 这些 Python 的 WEB 框架包装一下，几分钟就能搭出 REST 服务。
- 想接入飞书/钉钉？官方 SDK 已经提供了接收消息、发送消息的接口，你只需要在回调函数里调用你的 Agent。

LangChain 把表现层完全留给我们，因为它知道：每个项目的交互场景都不同，一刀切的方案反而会成为束缚。

2.3.5 【总结：LangChain的定位】

所以，在这里我们再总结一下 LangChain 在 Agent 三层架构中的角色：

层级	LangChain的贡献
能力层	提供了模型适配器、工具集、记忆模块，大幅减少重复代码
业务层	提供了Agent执行器、ReAct循环、LangGraph编排，简化决策
表现层	不干预，完全由开发者自由实现，保持灵活性

由此可知，LangChain 不是‘全包’的框架，而是一个‘积木盒’——它把 Agent 开发中最通用、最重复的部分封装好，让我们专注于业务逻辑和用户体验。

2.3.6 【实战】

有了这三层的认知，加上 LangChain 的辅助，我们现在可以开始动手了。

在接下来的实战环节，我会带大家：

1. 首先从能力层开始，定义工具和模型
2. 然后搭建业务层，用 `create_agent` 函数创建 Agent
3. 最后实现一个简单的命令行表现层，最终让 Agent 真正跑起来

大家可以跟着我一步一步来，争取在课程结束前，每个人都跑通自己的第一个 Agent。

2.4 工程实践中的注意事项

2.4.1 【开场：代码之外的“隐形功夫”】

上面三层架构讲完了，LangChain 的角色定位也讲清楚了。

下面在正式写代码之前，我想先插几句‘题外话’——但这几句题外话，恰恰是区分‘Demo Agent’和‘生产级应用 Agent’的关键。

很多同学写 Agent 的时候，脑子里只有功能：比如工具怎么调、记忆怎么存等等。

但往往忽略一些‘隐形功夫’——比如：环境变量、日志、错误处理、成本控制等等。

这些隐形功能看似琐碎，但如果我们不注意，上线之后就可能让我们头疼到失眠。

今天我就把曾踩过的坑提前告诉大家，咱们从一开始就把这些基础打牢。

2.4.2 【建议一：环境变量管理——别把密钥硬编码】

第一条：环境变量管理。

这是一个老生常谈但永远有人会犯的错误，比如：把API密钥直接写在代码里。

代码块

```
1  # 错误示范
2  api_key = "sk-1234567890abcdef"
```

这么做的问题是什么？一旦你的代码被上传到 GitHub（哪怕只是私有仓库），密钥就可能泄露。更可怕的是，如果你分享代码给别人，你的密钥就永远拱手送人了。

正确的做法：

- 用 `python-dotenv` 加载 `.env` 文件
- 把 `.env` 加入 `.gitignore`
- 代码里只读取环境变量。

如 PPT 截图所示：

代码块

```
1  from dotenv import load_dotenv
2  import os
3
4  load_dotenv()
5  api_key = os.getenv("OPENAI_API_KEY")
```

这样，密钥就只存在本地，谁也拿不走。如果是企业环境，还可以用配置中心（如Consul、Apollo）统一进行管理。总之一定要记住：密钥和代码，永远不要放在一起。

2.4.3 【建议二：日志记录——让Agent的每一步都“看得见”】

第二条：日志记录。

Agent执行过程是一个黑盒——模型在想什么？它调用了哪个工具？工具返回了什么？如果出了问题，你很难排查。

LangChain 集成了 LangSmith，这是一个可视化追踪平台，可以记录 Agent 的每一步思考、每一次工具调用、每一轮的输入输出。你只需要加几行配置：

代码块

```
1 from langsmith import Client
2
3 # 设置环境变量
4 os.environ["LANGCHAIN_TRACING_V2"] = "true"
5 os.environ["LANGCHAIN_API_KEY"] = "your_key"
```

之后每次调用 Agent，LangSmith 都会自动记录完整的执行轨迹，你可以像看回放一样查看每一步的细节。这对于调试、优化、审计都非常有用。

如果没有 LangSmith，至少也要用 Python 的 `logging` 模块把关键信息打印出来——比如‘调用了工具X，返回了Y’、‘进入第几轮思考’等。不要等到 Agent 跑飞了，才后悔没有记录日志。

2.4.4 【建议三：错误处理——别让一个小错误拖垮整个流程】

工程化建议三：做好错误处理。

Agent 在执行过程中可能遇到各种异常：API 超时、工具调用失败、模型返回格式错误等等……如果我们不对这些异常进行处理，Agent 可能会直接崩溃，或者陷入死循环。

一个健壮的 Agent 应该具备重试机制和降级策略。

例如，当我们调用天气 API 超时，可以重试 2 次，每次间隔 1 秒；如果仍然失败，就返回一个友好的提示，而不是抛出一串错误栈给用户。

在 LangChain 中，你可以用 `@tool` 装饰器在函数内部捕获异常并返回错误信息，而不是让异常向上抛出。或者使用 `tenacity` 库实现自动重试，如屏幕所示：

```
1 from tenacity import retry, stop_after_attempt, wait_fixed
2
3 @retry(stop=stop_after_attempt(3), wait=wait_fixed(1))
4 def call_api():
5     # 可能失败的调用
6     pass
```

我们需要异常处理：你的 Agent 应该永远 ‘优雅地失败’，而不是 ‘粗暴地崩溃’。

2.4.5 【建议四：成本控制——别让Agent “烧” 掉你的预算】

第四条：成本控制。

这是很多新手容易忽视的问题。Agent 循环可能会多次调用 API——一个复杂的任务，可能经过10轮思考-行动-观察，每一轮都要调用模型。如果用的是 GPT-4，一次复杂任务的成本可能轻松超过1美元。

那么我们应该如何控制成本？

- 设置最大迭代次数：在 `create_agent` 中通过 `max_iterations` 参数限制，比如最多15步，防止无限循环。
- 监控token消耗：LangChain 的 `Callback` 可以统计每次调用的 token 数，你可以在日志里记录，或者在超过阈值时自动停止。
- 选择合适的模型：简单任务用便宜的模型（如GPT-3.5或国产模型），只有复杂推理才用GPT-4等。LangChain支持多模型路由，可以自动选择。
- 缓存重复请求：如果用户反复问同一个问题，那我们可以缓存结果，避免重复调用API。

大家记住：Agent 的 ‘智能’ 是有成本的。提前规划，才能避免月底收到天价账单。

第三部分：实战1——基于LangChain的第一个Agent

3.1 环境准备与项目初始化

3.1.1 【从 “图纸” 到 “工地”】

好，蓝图有了，工具清楚了，工程化建议也介绍了。

下面，我们正式进入 ‘工地’，开始搭建第一个 Agent。

第一个案例比较简单，不需要复杂的 IDE。

请大家打开终端，跟着我一步一步来。我每敲一行命令，都会解释为什么要这么做。

3.1.2 【第一步：创建项目目录】

首先，我们要为这个项目创建一个独立的目录。个人建议不要在乱七八糟的文件夹里写代码，以后会找不到。

在终端中执行：

代码块

```
1  mkdir -p lab/2-quickstart
2  cd 2-quickstart
```

`mkdir` 是创建目录，`cd` 是进入这个目录。之后所有操作都在这下面进行。

为什么我们要建立独立目录？因为每个项目都有自己的依赖，混在一起容易冲突。就像一般我们不会把火锅底料和冰淇淋放在同一个冰箱层里。

3.1.3 【第二步：创建虚拟环境】

接下来，我们创建虚拟环境。虚拟环境是 Python 项目的最佳实践，它能隔离不同项目的依赖包，避免版本冲突。

我们使用 `venv` 模块来创建虚拟环境：

代码块

```
1  python -m venv venv
```

指令执行成功后，会在当前目录下创建一个叫 `venv` 的文件夹，该文件夹里面存放的就是独立的 Python 环境。

创建完成后，需要激活虚拟环境。激活后，你安装的 python 依赖就只会装在这个环境里，不会影响系统全局。

不同操作系统使用的激活指令是不一样的，大家根据各自的操作系统，选择对应的环境激活指令。

- Mac / Linux：

代码块

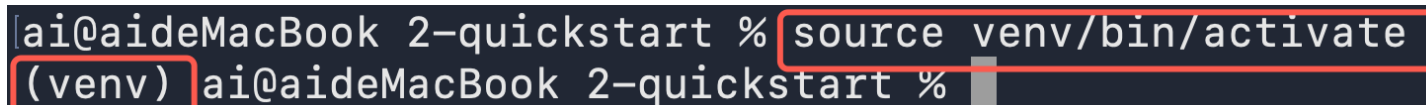
```
1 source venv/bin/activate
```

- Windows:

代码块

```
1 venv\Scripts\activate
```

激活成功后，你的终端提示符前面会出现 `(venv)`，说明你已经进入了虚拟环境。



```
ai@aideMacBook 2-quickstart % source venv/bin/activate
(venv) ai@aideMacBook 2-quickstart %
```

注意：如果 `python` 命令不行，试试 `python3`。不同系统可能默认Python版本不同。

3.1.4 【第三步：安装依赖包】

环境准备好了，现在安装我们今天需要的三个核心包：

代码块

```
1 pip install langchain langchain-openai python-dotenv
```

- `langchain`：核心框架，提供 Agent 的工具、记忆等能力。
- `langchain-openai`：OpenAI模型适配器。
- `python-dotenv`：用来从 `.env` 文件加载环境变量，避免密钥硬编码。

安装过程可能需要几秒到一分钟。如果下载慢，大家可以切换成国内的镜像源，比如：

代码块

```
1 pip install langchain langchain-openai python-dotenv -i
  https://mirrors.aliyun.com/pypi/simple/
```

依赖安装成功后截图如下：


```
d3c70107e216ead6/jiter-0.14.0-cp311-cp311-macosx_11_0_arm64.whl (321 kB)
Collecting sniffio (from openai<3.0.0,>=2.26.0->langchain-openai)
  Downloading https://mirrors.aliyun.com/pypi/packages/e9/44/75a9c9421471a6c4805dbf2356f7c181a29c1879239a
bab1ea2cc8f38b40/sniffio-1.3.1-py3-none-any.whl (10 kB)
Collecting tqdm>4 (from openai<3.0.0,>=2.26.0->langchain-openai)
  Downloading https://mirrors.aliyun.com/pypi/packages/16/e1/3079a9ff9b8e11b846c6ac5c8b5bfb7ff225eee72182
5310c91b3b50304f/tqdm-4.67.3-py3-none-any.whl (78 kB)
Collecting regex>=2022.1.18 (from tiktoken<1.0.0,>=0.7.0->langchain-openai)
  Downloading https://mirrors.aliyun.com/pypi/packages/eb/3b/637181b787dd1a820ba1c712cee2b4144cd84a32dc77
6ca067b12b2d70c8/regex-2026.4.4-cp311-cp311-macosx_11_0_arm64.whl (289 kB)
Collecting charset_normalizer<4,>=2 (from requests>=2.0.0->langsmith<1.0.0,>=0.3.45->langchain-core<2.0.0
,>=1.2.10->langchain)
  Downloading https://mirrors.aliyun.com/pypi/packages/c2/d7/b5b7020a0565c2e9fa8c09f4b5fa6232feb326b8c200
81ccded47ea368fd/charset_normalizer-3.4.7-cp311-cp311-macosx_10_9_universal2.whl (309 kB)
Collecting urllib3<3,>=1.26 (from requests>=2.0.0->langsmith<1.0.0,>=0.3.45->langchain-core<2.0.0,>=1.2.1
0->langchain)
  Downloading https://mirrors.aliyun.com/pypi/packages/39/08/aaaad47bc4e9dc8c725e68f9d04865dbcb2052843ff0
9c97b08904852d84/urllib3-2.6.3-py3-none-any.whl (131 kB)
Installing collected packages: zstandard, xxhash, uuid-utils, urllib3, typing-extensions, tqdm, tenacity,
sniffio, regex, pyyaml, python-dotenv, packaging, ormsgpack, orjson, jsonpointer, jiter, idna, h11, dist
ro, charset_normalizer, certifi, annotated-types, typing-inspection, requests, pydantic-core, jsonpatch,
httpcore, anyio, tiktoken, requests-toolbelt, pydantic, httpx, openai, langsmith, langgraph-sdk, langchai
n-core, langgraph-checkpoint, langchain-openai, langgraph-prebuilt, langgraph, langchain
Successfully installed annotated-types-0.7.0 anyio-4.13.0 certifi-2026.2.25 charset_normalizer-3.4.7 dist
ro-1.9.0 h11-0.16.0 httpcore-1.0.9 httpx-0.28.1 idna-3.12 jiter-0.14.0 jsonpatch-1.33 jsonpointer-3.1.1 l
angchain-1.2.15 langchain-core-1.3.0 langchain-openai-1.1.16 langgraph-1.1.9 langgraph-checkpoint-4.0.2 l
anggraph-prebuilt-1.0.10 langgraph-sdk-0.3.13 langsmith-0.7.33 openai-2.32.0 orjson-3.11.8 ormsgpack-1.12
.2 packaging-26.1 pydantic-2.13.3 pydantic-core-2.46.3 python-dotenv-1.2.2 pyyaml-6.0.3 regex-2026.4.4 re
quests-2.33.1 requests-toolbelt-1.0.0 sniffio-1.3.1 tenacity-9.1.4 tiktoken-0.12.0 tqdm-4.67.3 typing-ext
ensions-4.15.0 typing-inspection-0.4.2 urllib3-2.6.3 uuid-utils-0.14.1 xxhash-3.6.0 zstandard-0.25.0
(venv) ai@aideMacBook 2-quickstart %
```

3.1.5 【第四步：创建代码和配置文件】

依赖安装完后，我们创建两个文件：

代码块

```
1 touch lc_agent_demo.py .env
```

- `lc_agent_demo.py`：这个文件是我们的主程序代码。
- `.env`：存放 API 密钥等环境变量的配置文件。

这里再强调下：`.env` 文件千万不要提交到Git仓库！一定要把它加入 `.gitignore`。因为密钥一旦泄露，你的账户可能被盗刷。

现在我们用编辑器打开 `.env` 文件，写入你的API密钥。比如用OpenAI：

代码块

```
1 OPENAI_API_KEY=sk-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

如果你用的是国产模型（如智谱、阿里通义），可以写对应的变量名，比如：

```
DASHSCOPE_API_KEY=sk-xxxxxxxxxxxxxxxxxxxxx
```

注意：密钥两边不要加引号，直接写值即可。

3.1.6 【第五步：验证环境是否就绪】

在写正式代码之前，我们先快速验证一下环境是否配置正确。

在 `lc_agent_demo.py` 里写几行测试代码：

```
lc_agent_demo.py
1  import os
2  from dotenv import load_dotenv
3  from langchain_openai import ChatOpenAI
4
5  load_dotenv()
6
7  api_key = os.getenv("DASHSCOPE_API_KEY")
8  print(f"API Key loaded: {api_key[:5]}...") # 只显示前5位，安全第一
9
10 model = ChatOpenAI(model="qwen-plus", api_key=api_key,
    openai_api_base="https://dashscope.aliyuncs.com/compatible-mode/v1")
11 response = model.invoke("Hello, 测试一下")
12 print(response.content)
```

保存后运行：

代码块

```
1 python lc_agent_demo.py
```

如果看到类似 `API Key loaded: sk-xx...` 和模型返回的问候语，说明环境OK。

如果报错：

- `ModuleNotFoundError`：说明某个包没装上，重新 `pip install`。
- `AuthenticationError`：说明API密钥错误或无效。
- `ConnectionError`：检查网络，或换个镜像源。

这一步是确保后续代码能正常运行，不要急着跳过。

3.1.7 【准备就绪，开始写代码】

好，环境已经就绪，依赖安装完毕，密钥也都配置好了。现在我们的‘工地’已经搭好了脚手架，可以开始砌墙了。

接下来，我们将按照三层架构，从能力层开始，一步步构建我们的第一个 Agent。

大家检查一下，确认终端里能看到 `(venv)` 提示符，并且刚才的测试代码运行成功。如果有任何问题，可以联系我们的助教老师，我们会协助您一起解决。

没问题的话，我们就进入下个环节：定义能力层——工具与模型。

3.2 创建最简单的 Agent

3.2.1 【从“能力层”到“业务层”】

接下来我们要写的这个 Agent，暂时不带任何工具，只做对话，也就是一个简单的聊天型 Agent。

别小看这一步，它能让你理解 Agent 的最简形态，也为后面扩展打下基础。

首先，大家新创建一个名为 `lc_chat_agent.py` 的文件，然后开始代码编写。

3.2.2 【第一步：导入依赖和加载环境变量】

代码开始，在我们文件顶部导入所需要的 python 库：

代码块

```
1 import os
2 from dotenv import load_dotenv
3 from langchain_openai import ChatOpenAI
4 from langchain.agents import create_agent
```

- `os`：用来读取环境变量。
- `load_dotenv`：从 `.env` 文件加载变量。
- `ChatOpenAI`：OpenAI 模型的适配器，LangChain 已经封装好。

- `create_agent`：LangChain 中统一的智能体 Agent 创建函数。它内部会自动构建 ReAct 模式的执行图，并支持传入工具、系统提示等参数。

接下来的指令用来加载 `.env` 文件中的变量：

代码块

```
1 load_dotenv()
```

这行代码会把 `.env` 里的键值对注入到环境变量中。之后我们就可以用 `os.getenv()` 函数读取配置文件中的环境变量了。

3.2.3 【第二步：初始化模型——能力层的核心】

接下来，我们创建模型实例。这是能力层的核心组件之一。

代码块

```
1 model = ChatOpenAI(  
2     model="qwen-plus", # 模型名称，可以用gpt-4、gpt-3.5-turbo，或者国产模型如"qwen-  
    plus"  
3     temperature=0.7, # 控制随机性，0.7比较平衡  
4     api_key=os.getenv("DASHSCOPE_API_KEY"),  
5     openai_api_base="https://dashscope.aliyuncs.com/compatible-mode/v1")
```

- `model` 参数：指定模型名称。如果你用的是 OpenAI，常见的有 `gpt-4`、`gpt-3.5-turbo`。如果想用国产模型，这里可以换成对应的模型 ID，比如智谱的 `glm-4`、阿里的 `qwen-plus` 等，我们这里采用的是阿里的 `qwen` 系列。
- `temperature`：该参数用来控制模型输出的随机性。0 表示最确定，1 表示最随机。0.7 是个常用值，既能保持一定创意，又不会太离谱，简单记法就是温度越高，越天马行空。
- `api_key`：从环境变量读取模型的 API KEY，安全。
- `openai_api_base`：这是 `qwen` 模型 API URL 网址。

为什么要把模型单独拿出来？因为这是能力层的重要抽象——模型适配器。将来你想换模型，只需要改这里，业务层完全不用动。

我们可以先测试一下模型是否正常工作。你可以临时加一行：

代码块

```
1 response = model.invoke("你好，请用一句话介绍你自己")  
2 print(response.content)
```

运行看看，如果能输出内容，说明模型连接成功。

执行成功后截图如下：

```
(venv) ai@aideMacBook 2-quickstart % python lc_chat_agent.py
你好！我是通义千问（Qwen），阿里巴巴集团旗下的超大规模语言模型，能够回答问题、创作文字，如写故事、公文、邮件、剧本等，还能进行逻辑推理、编程，甚至表达观点和玩游戏，支持多种语言，致力于为你提供高效、友好的帮助。
```

3.2.4 【第三步：创建Agent——业务层的入口】

模型有了，现在我们来创建 Agent。

代码块

```
1 agent = create_agent(
2     model=model,
3     tools=[], # 暂时没有工具
4     system_prompt="你是一个友好的AI助手，用简洁易懂的语言回答用户问题。"
5 )
```

`create_agent` 是 LangChain 框架中创建 Agent 的统一接口。它接收三个核心参数：

- `model`：刚才创建的模型实例。
- `tools`：工具列表，目前为空，后面我们会加。
- `system_prompt`：系统提示词，告诉 Agent 它的角色和行为准则。

这里我们写的系统提示词是‘你是一个友好的AI助手，用简洁易懂的语言回答用户问题。’——这会让 Agent 回答问题时更亲切，不会太学术。

注意：`create_agent` 内部会自动构建一个 ReAct 模式的执行图，包括思考-行动-观察的循环。即使没有工具，它也会维持这个结构，只是行动部分只是‘回答’，而不会调用工具，因为我们这里的 `tools` 参数并没有添加工具。

3.2.5 【第四步：执行对话——表现层最简单的实现】

Agent 创建好了，现在我们来执行一次对话。这是最简单的表现层——直接调用 Agent 并打印结果。

代码块

```
1 response = agent.invoke({
```

```
2     "messages": [{"role": "user", "content": "介绍一下什么是AI Agent"}]  
3 }
```

`agent.invoke` 函数是 LangChain 中通用的执行方法。这里我们传入一个字典，里面有一个 `messages` 列表，包含用户消息。LangChain 内部会把这条消息交给模型，然后模型会返回完整的消息历史。

注意，`invoke` 返回的结果也是一个字典，其中 `messages` 键包含了所有消息。最后一条消息就是 Agent 的最终回答，我们可以这样取出：

代码块

```
1 print(response["messages"][-1].content)
```

`[-1]` 表示最后一条，`content` 取内容。

现在，我们保存文件，在终端运行：

代码块

```
1 python lc_chat_agent.py
```

你应该会看到 Agent 输出一段关于 AI Agent 的介绍，比如：

‘AI Agent 是一种能够自主规划、执行任务的人工智能系统。它通过大语言模型理解意图，结合工具调用、记忆和规划能力，完成复杂的工作流……’

下面就是我执行成功后的控制台输出：

```
(venv) root@localhost:~/zhibo/my_first_agent# python agent_demo.py  
API Key loaded: sk-78...  
AI Agent (人工智能代理) 是一种**能自主感知环境、理解目标、规划行动并执行任务的智能系统**。它不只是被动响应指令，而是像一个“数字助手”或“虚拟员工”，具备一定的**自主性、推理能力和工具使用能力**。  
  
✅ 核心特点：  
1. **自主性 (Autonomy)**  
能在无人实时干预下，根据目标主动决策和行动（比如：自动订机票、写报告、调试代码）。  
2. **感知与理解 (Perception & Reasoning)**  
通过输入（文字、语音、图像等）理解用户意图，并结合上下文、知识库或记忆进行推理。  
3. **规划与决策 (Planning)**  
把复杂目标拆解为多步任务（例如：“帮我准备一场关于气候变化的中学讲座” → 搜资料 → 提纲 → 做PPT → 生成讲稿）。  
4. **工具调用 (Tool Use)**  
能调用外部能力，如搜索网页、查数据库、运行代码、调用API、操作软件等（不只靠大模型“脑补”，而是“动手做事”）。  
5. **记忆与学习 (Memory & Adaptation)**  
可短期记住对话上下文，也可长期保存用户偏好/历史经验，逐步提升表现（部分Agent支持微调或反馈优化）。  
  
💡 举个生活化例子：  
你对AI Agent说：“帮我查下周北京到上海的低价航班，选早班机，订好后发日历提醒，并同步给我的团队。”  
→ 它会自动：搜索航班 → 筛选时间与价格 → 调用购票接口（或生成确认信息） → 创建日历事件 → 发邮件/消息通知团队。全程无需你一步步指挥。  
  
💡 和传统AI的区别：  
- ❌ 普通聊天机器人（如基础Chatbot）：只能“回答问题”，无法主动执行、调用工具或跨步骤协作。  
- ✅ AI Agent：是“做事的AI”，强调**目标导向 + 行动闭环**。  
  
🔍 当前典型形态：  
- 基于大语言模型 (LLM) 构建的智能体框架（如LangChain、LlamaIndex、AutoGen、Microsoft AutoGen、Google Gemini Agents）  
- 操作系统级Agent（如苹果iOS 18的Apple Intelligence、华为盘古Agent）  
- 垂直场景Agent：编程助手（GitHub Copilot X）、科研Agent、客服Agent、金融投顾Agent等  
  
🔥 简单一句话总结：  
> **AI Agent = 大模型（大脑） + 规划能力（导航仪） + 工具调用（手脚） + 记忆（笔记本） → 能独立完成任务的智能执行者。**  
  
需要的话，我还可以帮你设计一个简单的Agent流程，或者对比几个主流Agent框架 🌟
```

虽然我们没给工具，但模型本身的知识就足以回答这个问题。

3.2.6 【第五步：讲解 Agent 的内部机制】

看到输出结果，我们来聊聊背后发生了什么。

当我们调用 `agent.invoke` 时，LangChain 内部会做以下几件事：

1. 构建消息列表：将用户消息和系统提示组合，形成完整的对话上下文。
2. 调用模型：将消息发送给模型，模型返回回答。
3. 判断是否需要工具：因为我们的工具列表为空，模型不会尝试调用工具，直接生成最终回答。
4. 返回结果：将模型的回答包装成 `response` 字典。

虽然这个流程很简单，但它已经具备了 Agent 的基本框架：消息输入 → 模型推理 → 输出。后面我们加上工具，就会触发更复杂的循环：模型会输出工具调用指令，LangChain 自动执行工具，把结果传回模型，模型再继续思考。

这里有一个关键点：`create_agent` 默认使用的是 ReAct 模式。即使没有工具，它也会把‘回答’当作一种特殊的行动。所以，这个 Agent 已经具备了‘思考-行动-观察’的循环雏形。

3.2.7 【互动】

也许有同学会问：‘为什么这里不直接用 `model.invoke` 方法来调用，为什么非要绕一圈用 Agent?’ 来调用？

——这真是一个好问题！原因在于：如果直接使用 `model.invoke` 方法确实可以让模型回答问题，但这只适合单轮对话。而我们使用 Agent 的真正价值和目的在于让它支持多轮交互和工具调用。

比如，如果你直接用 `model.invoke`，问‘现在几点？’，它只会告诉你‘我不知道，我没有实时数据’。但如果我们给 Agent 加上时间工具，它就能自己调用工具、获取实时时间、然后回答。这种‘自主决策’的能力，是普通模型调用做不到的。

3.2.8 【为下一步做准备】

好了，现在我们已经成功创建了第一个智能体 Agent——虽然它没有工具，但它已经能回答问题、理解系统提示。

接下来，我们给它装上‘手脚’，这里我们会添加两个工具：分别是获取当前时间工具和简单计算器工具。

这样，它就能真正干活了。

接下来，我们会在现有代码基础上，为智能体 Agent 定义工具，并传入 `tools` 列表，让 Agent 学会调用它们。

3.3 添加第一个工具：获取当前时间

3.3.1 【让Agent长出“手脚”】

好，刚才我们创建了一个最基础的 Agent——它能对话，能回答问题，但它有一个明显的缺陷：它没有实时信息。

比如你问它‘现在几点’，它会怎么回答？它可能会说‘我没有实时时钟，无法获取当前时间’。因为它只能依赖训练数据里的知识，而训练数据是几个月前的。效果如下图所示：

```
(venv) root@localhost:~/zhibo/my_first_agent# python agent_demo.py
'API Key loaded: sk-78...'
我无法实时获取当前的日期和时间。你可以查看你设备（手机、电脑等）右下角或顶部的状态栏，那里会显示准确的日期和时间 😊
需要我帮你计算某个日期相关的问题吗？
```

这就是 Agent 和普通聊天机器人的第一个分水岭：能不能调用外部工具获取实时信息。

下面，我们就给 Agent 装上第一只‘手脚’——一个获取当前时间的工具。有了它，Agent 就能回答‘现在几点’、‘今天日期’这类问题了。

大家跟着我一起，在刚才的代码基础上继续扩展功能。

3.3.2 【第一步：导入依赖和定义工具】

首先，我们需要导入 `tool` 装饰器和 `datetime` 模块。我们在文件顶部添加：

代码块

```
1  import os
2  from dotenv import load_dotenv
3  from langchain_openai import ChatOpenAI
4  from langchain.agents import create_agent
5  from langchain_core.tools import tool
6  from datetime import datetime
```

`tool` 是 LangChain 提供的装饰器，它能将一个普通函数变成 Agent 可以调用的工具。`datetime` 是 Python 标准库，用来获取系统时间。

然后，我们在 `create_agent` 之前定义这个工具函数：


```

1 @tool
代码块
2 def get_current_time():
3     """返回当前的日期和时间。当用户问'现在几点'、'今天日期'时调用此工具。"""
4     now = datetime.now()
5     return now.strftime("%Y-%m-%d %H:%M:%S")

```

这个函数很简单：用 `datetime.now()` 函数获取当前时间，然后把该时间格式化成 `年-月-日 时:分:秒` 的字符串并返回。

注意看这个文档字符串说明——`"""返回当前的日期和时间。当用户问'现在几点'、'今天日期'时调用此工具。"""`。这个不是普通的注释，它会被 LangChain 提取并传给模型。模型会根据这个描述来判断什么时候调用这个工具。

所以，写清楚工具的描述至关重要。描述越精准，模型调用就越准确。

3.3.3 【第二步：将工具添加到Agent】

工具定义好了，接下来我们要把它注册给 Agent 中。

我们修改原来代码中 `create_agent` 函数所在的那一行，在 `tools` 参数里传入这个获取当前时间的工具，还记得不，以前这个 `tools` 参数我们并没有赋值：

代码块

```

1 agent = create_agent(
2     model=model,
3     tools=[get_current_time], # 这里！把工具放进列表
4     system_prompt="你是一个友好的AI助手，当需要获取实时时间时，请调用get_current_time
工具。"
5 )

```

注意，`tools` 参数接收的是一个列表。因此我们这里可以同时传入多个工具，目前在这里我们先增加一个。

同时，我稍微修改了系统提示词，明确告诉 Agent ‘当需要获取实时时间时，请调用这个工具’。虽然模型有能力自己判断，但显式提示可以增加准确性和可靠性。

现在，Agent 的配置已经完备：它知道有一个工具叫 `get_current_time`，它的作用是什么，什么时候该用它。

3.3.4 【第三步：测试工具调用

接下来，我们测试一下。把刚才的用户消息改成问时间：

代码块

```
1 response = agent.invoke({
2     "messages": [{"role": "user", "content": "现在几点了? "}])
3 })
4 print(response["messages"][-1].content)
```

保存文件，然后在终端运行：

代码块

```
1 python agent_demo.py
```

你会看到类似这样的输出：

代码块

```
1 现在是2026年3月20日 15:22:08。
```

代码执行成功截图如下：

```
(venv) root@localhost:~/zhibo/my_first_agent# python agent_with_tools_demo.py
API Key loaded: sk-78...
现在是2026年3月22日，下午5点26分41秒。
```

我们发现程序成功了！而且回答符合我们的预期，Agent 不仅理解了‘现在几点了’，还会自动调用我们定义的工具，获取了实时时间，最后把结果整合成一句友好的回答打印到了屏幕上。

那么如果我们再问别的问题呢？比如问‘你好吗？’，我们发现此时 Agent 它并没有调用工具，只会直接回答。因为模型判断这个问题并不需要工具调用。

这就是 Agent 的智能所在：它会根据你的问题，自己决定到底该不该调用工具，什么时候不需要，模型会基于语义理解自主决策。

3.3.5 【深度解析：@tool装饰器与工具调用原理】

程序已经跑通了，接下来我们来深挖一下 LangChain 实现 Agent 的背后原理。经过分析，主要有以下两点：

第一点：@tool 装饰器做了什么？

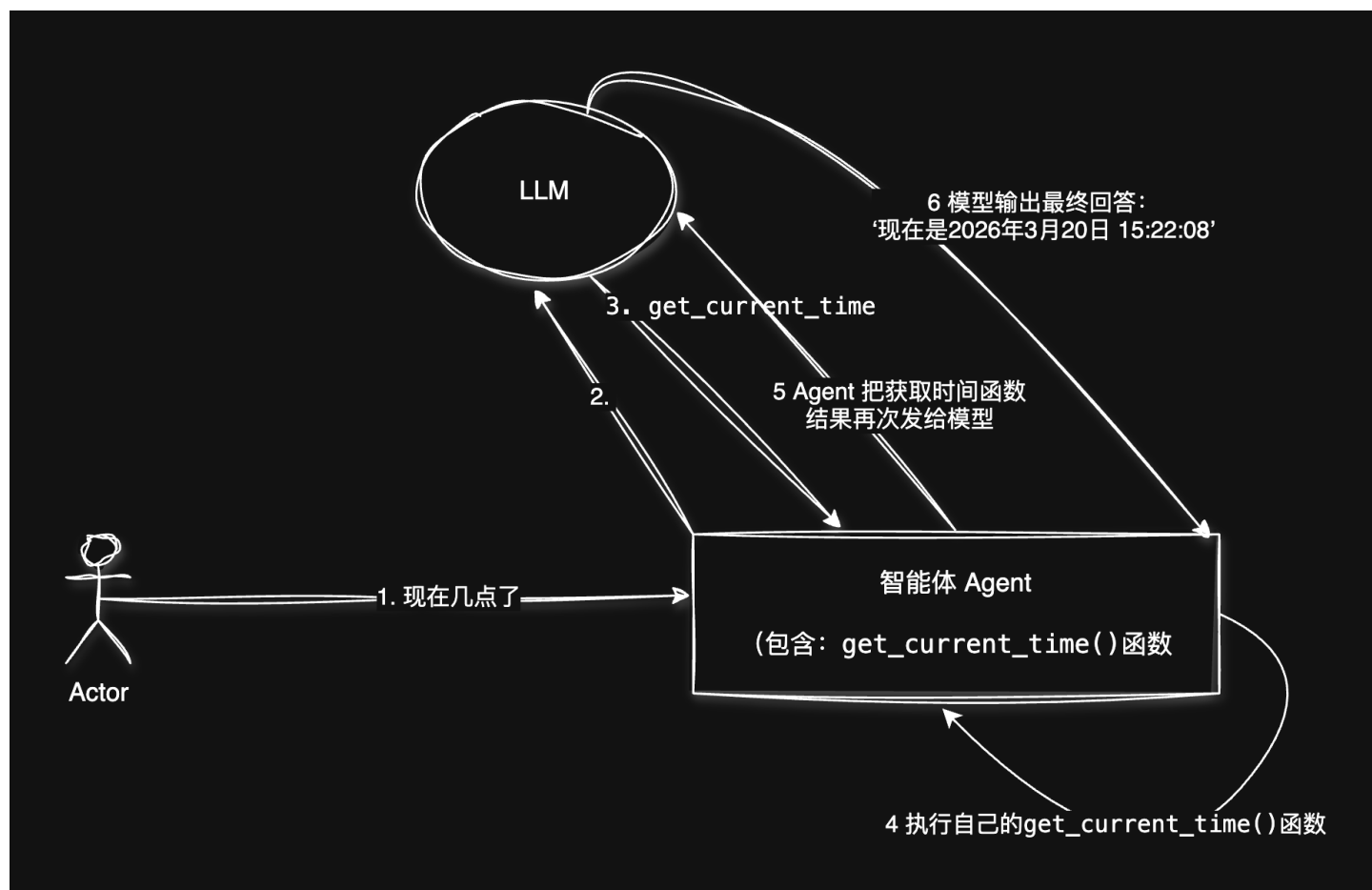
当你写 `@tool` 时，LangChain 会把这个函数包装成一个 `StructuredTool` 对象。它会提取函数的签名、参数、文档注释字符串，并生成一份对模型的描述。经过 `@tool` 标注的函数经过 LangChain 处理后，传给模型的工具描述大致是以下这个格式的：

代码块

```
1  {
2    "name": "get_current_time",
3    "description": "返回当前的日期和时间。当用户问'现在几点'、'今天日期'时调用此工具。",
4    "parameters": {
5      "type": "object",
6      "properties": {},
7      "required": []
8    }
9  }
```

模型看到这个描述后，就知道：有一个工具函数叫 `get_current_time`，这个函数不需要参数，它的作用是返回当前时间。当用户问时间相关问题时，模型就像出于自然反应，在输出中生成一个工具调用指令。

第二点：我们再来看下工具调用的完整流程：



1. 当用户说 ‘现在几点了’。
2. Agent 把用户消息和系统提示、工具函数描述一起发给模型。

3. 模型分析后，决定调用 `get_current_time`，并输出一个结构化的调用指令。
4. LangChain 的 Agent 执行器捕获这个指令，立即执行我们自己编写的 `get_current_time()` 函数。
5. 函数返回结果（如 ‘2026-03-20 15:22:08’ ）。
6. Agent 把调用函数的结果再次发给模型，让模型整合成自然语言回复。
7. 模型输出最终回答： ‘现在是2026年3月20日 15:22:08’ 。

这个过程就是 ReAct 循环中的 ‘思考-行动-观察’ 环节。我们刚才看到的只是一次调用，但 Agent 内部实际发生了两次模型调用：第一次让模型 **决策要不要调用工具**，第二次让模型 **生成最终回复**。

3.3.6 【互动】

也许有学员会打破砂锅问到底，如果我们问：今天星期几，这个工具能回答吗？

—— **不一定**。因为我们定义的工具只返回日期和时间，并没有返回星期几。模型拿到 ‘2026-03-22’ 后，它自己知道这天是星期日吗？不一定。如果模型训练数据里没有这个知识，它可能乱猜。

所以，工具设计要精准匹配需求。如果你需要星期几，要么在工具里返回星期信息，要么再写一个专门的工具。这就是为什么 Agent 开发需要仔细设计工具集。

也许还有学员会问，如果把工具描述写错了会怎样？

——比如如果描述说 ‘当用户问天气时调用’，但用户问时间，模型有可能就不会调用，或者错误调用。因此工具描述的质量将直接影响 Agent 的准确性，这就要求我们在写描述时务必具体、准确，把触发场景说清楚。

第四部分：实战2——集成Tools+Memory

4.1 添加第二个工具：简单计算器

4.1.1 【开场：从 “时间” 到 “计算” 】

好，刚才我们给 Agent 装上了第一只手——获取时间。现在，我们要给它装上第二只手：计算器。

为什么需要计算器？因为在实际场景中，用户经常会问一些需要计算的问题，比如 ‘23乘以45加100等于多少’、‘我下个月工资扣税后还剩多少’。模型虽然有一定的数学能力，但复杂计算容易出错，而且它不会真正 ‘算’，只是 ‘猜’。

所以，我们让 Agent 调用一个真正的计算器工具，然后把计算任务交给 Python，这样既准确又高效。现在，我们在刚才的代码基础上，继续添加第二个工具。

4.1.2 【第一步：定义计算器工具】

打开文件，在 `get_current_time` 函数后面，新增一个新工具——计算器工具：

代码块

```
1  @tool
2  def calculator(expression: str):
3      """计算数学表达式。输入应该是一个字符串形式的数学表达式，如'2+3*4'。"""
4      try:
5          # 安全地计算表达式
6          print("执行自定义算数运算")
7          result = eval(expression, {"__builtins__": {}}, {})
8          return f"计算结果: {expression} = {result}"
9      except Exception as e:
10         return f"计算失败: {str(e)}"
```

注意，这个工具有一个参数： `expression: str`。

这和上一个工具不同—— `get_current_time` 没有参数，而这个工具需要接收一个表达式字符串。LangChain 会根据函数的签名自动生成参数描述，告诉模型：‘调用这个工具时，需要传入一个字符串参数，叫做 `expression`’。

模型在决定调用时，会从用户的问题中提取出表达式，比如用户说‘帮我算一下 $23 \times 45 + 100$ ’，模型就会把“ $23 \times 45 + 100$ ”作为参数传进来。

关于 `eval` 的安全性问题：我用了 `eval`，我知道大家会担心安全问题。`eval` 可以执行任意代码，如果用户输入 `import('os').system('rm -rf /')`，你的电脑直接就被完爆了。

所以在这里，我加了两层保护：

- 限制 `builtins` 为空，让 `eval` 不能调用任何内置函数。
- 传入空的全局和局部字典，进一步限制。

但说实话，这只是‘演示级’的安全。在生产环境中，绝对不要用 `eval` 函数！应该用 `ast.literal_eval`（只能处理字面量）或 `numexpr` 库，或者用专门的数学表达式解析库，比如 `sympy` 或 `pyexpression`。

为什么这里我要用 `eval` 演示？因为它最直观，让你一眼看懂工具函数的逻辑。但在实际项目中，大家一定要替换成安全的实现。

4.1.3 【第二步：参数传递机制详解】

我们来深入看看 LangChain 是怎么处理这个参数的。

当你用 `@tool` 装饰器包装 `calculator(expression: str)` 时，LangChain 会自动做几件事：

1. 解析函数签名：发现有一个参数 `expression`，类型是 `str`。
2. 生成工具描述：模型会看到类似这样的JSON：

代码块

```
1  {
2    "name": "calculator",
3    "description": "计算数学表达式。输入应该是一个字符串形式的数学表达式，如'2+3*4'。",
4    "parameters": {
5      "type": "object",
6      "properties": {
7        "expression": {
8          "type": "string",
9          "description": "数学表达式，如'2+3*4'"
10       }
11     },
12    "required": ["expression"]
13  }
14 }
```

3. 模型调用时：模型会根据用户输入，决定调用 `calculator`，并生成一个参数值。比如用户说‘帮我算一下 23乘以45加100’，模型会输出：

代码块

```
1  {
2    "function_call": {
3      "name": "calculator",
4      "arguments": "{\"expression\": \"23*45+100\"}"
5    }
6  }
```

4. 执行：LangChain 解析这个JSON，然后调用 `calculator(expression="23*45+100")`，拿到结果。

整个过程对开发者是透明的——你只需要定义好函数签名和文档字符串，剩下的 LangChain 会帮你搞定。

这里有个关键点：参数名 `expression` 和类型提示 `: str` 非常重要。LangChain 依赖这些信息来生成正确的参数描述。如果参数名是 `x`，模型可能不知道要传什么；如果类型是 `int`，模型可能会把字符串表达式解析成整数，这就会导致失败。

所以，写工具函数时，参数名要语义清晰，类型提示要准确。

4.1.4 【第三步：测试计算器工具】

现在，我们把计算器工具加入Agent的 `tools` 列表：

代码块

```
1 agent = create_agent(  
2     model=model,  
3     tools=[get_current_time, calculator], # 两个工具  
4     system_prompt="你是一个友好的AI助手。当需要实时时间时调用get_current_time，当需要  
    数学计算时调用calculator。"  
5 )
```

然后测试一下：

代码块

```
1 response = agent.invoke(  
2     "messages": [{"role": "user", "content": "23乘以45加100等于多少? "}]  
3 )  
4 print(response["messages"][-1].content)
```

运行后，你应该看到类似：

代码块

```
1 计算结果: 23*45+100 = 1135
```

或者Agent会包装成一句完整的话：“23 乘以 45 加 100 等于 1135。”，如下图所示：

```
(venv) root@localhost:~/zhibo/my_first_agent# python agent_with_tools_demo.py  
API Key loaded: sk-78...  
23乘以45加100等于1135。
```

4.2 集成短期记忆

4.2.1 【没有记忆的Agent就像“金鱼”】

好，现在我们的 Agent 已经有两个工具了——能查时间、能算数。看起来挺能干，对吧？但有一个致命的缺陷：它没有记忆。

什么意思？你试试跟它说：‘我叫张三’。然后下一轮问：‘我叫什么名字？’它大概率会回答：‘我不知道，你没告诉我。’

因为它每次对话都是独立的，也就是每次程序调用模型 API 就开启了一轮新的会话，此时 Agent 就像一条金鱼，7 秒后就忘了刚才发生了什么。这显然不是我们想要的智能助手。

下面，我们就来解决这个问题——给 Agent 加上短期记忆，让它在同一会话内记住对话历史。这样它就能记住你的名字、偏好，甚至之前聊过的内容。

在 LangChain 中，实现记忆最强大的方式是 LangGraph 的检查点机制。下面我们一步步来实现。

4.2.2 【第一步：导入LangGraph相关模块】

首先，我们需要导入 LangGraph 的核心组件。在文件顶部添加：

代码块

```
1 from langgraph.checkpoint.memory import InMemorySaver
2 from langgraph.graph import StateGraph, END
3 from langgraph.graph.message import add_messages
4 from typing import TypedDict, Annotated
```

- `StateGraph`：LangGraph 的核心类，用来构建状态图。你可以把状态图理解为一个有状态的工作流，每一步都能访问和更新状态。
- `InMemorySaver`：一个简单的检查点存储，把状态保存在内存里。适合开发和测试，生产环境可以换成数据库存储（如PostgreSQL）。
- `add_messages`：一个辅助函数，用来合并消息列表，避免重复。
- `TypedDict`、`Annotated`：用来定义状态的结构。

有了这些，我们就可以开始构建带记忆的 Agent 了。

4.2.3 【第二步：定义状态结构】

LangGraph 的核心是状态。状态就像 Agent 的‘便签本’，可以在每一步之间传递信息。

我们先定义状态的结构：

代码块

```
1 class AgentState(TypedDict):
2     messages: Annotated[list, add_messages]
```

这个状态里只有一个字段：`messages`，它是一个消息列表。`Annotated[list, add_messages]` 的意思是：当我们更新状态时，`add_messages` 会自动把新消息追加到列表末尾，而不是覆盖。这样就能自然地累积对话历史。

你也可以在状态里放更多字段，比如用户偏好、任务进度等。但今天我们先只放消息。

4.2.4 【第三步：创建状态图并添加节点】

状态定义好了，接下来我们创建状态图：

代码块

```
1 graph = StateGraph(AgentState)
```

这创建了一个空的状态图，它的状态类型是 `AgentState`。

然后，我们定义一个节点函数。节点是状态图中的执行单元，每个节点接收当前状态，处理后返回更新后的状态。

代码块

```
1 def call_model(state: AgentState):
2     # 这里简化处理，实际应该用完整的Agent逻辑
3     response = model.invoke(state["messages"])
4     return {"messages": [response]}
```

在这个节点里，我们只做了一件事：调用模型，传入当前的所有消息（也就是历史对话），然后把模型的回答作为新消息返回。注意，返回的是一个字典，键是 `messages`，值是一个列表。

`add_messages` 会自动把这条新消息追加到状态中。

接下来，我们把节点添加到图中：

代码块

```
1 graph.add_node("agent", call_model)
2 graph.set_entry_point("agent")
3 graph.add_edge("agent", END)
```

- `add_node`：添加一个节点，名字叫 `agent`，对应函数 `call_model`。
- `set_entry_point`：设置入口节点，也就是从哪个节点开始执行。
- `add_edge`：添加一条边，从 `agent` 节点到 `END`，表示执行完就结束。

现在，这个图只有一个节点，执行顺序就是：开始 → `agent` → 结束。虽然简单，但它已经具备了状态管理的骨架。

4.2.5 【第四步：添加检查点实现记忆】

现在，最关键的一步：添加检查点。

代码块

```
1 checkpointer = InMemorySaver()
2 graph = graph.compile(checkpointer=checkpointer)
```

`InMemorySaver` 是一个检查点存储器，它会在每一步执行后保存当前状态。下次执行时，如果使用相同的 `thread_id`，就会自动加载之前的状态。

`graph.compile` 把图编译成可执行的运行器，并传入检查点。

有了检查点，状态就变成了持久化的。同一个会话的多次调用，可以通过 `thread_id` 关联起来。

这里要强调的是：`InMemorySaver` 只是存在内存里，进程重启就会丢失。生产环境可以用 `SqliteSaver` 或 `PostgresSaver`，把状态存到数据库，实现跨会话的长期记忆。

4.2.6 【第五步：使用thread_id实现记忆对话】

现在，我们来测试记忆功能。首先，定义一个 `config`，里面包含 `thread_id`。这个ID标识了一个会话，同一个ID的所有调用共享状态。

代码块

```
1 config = {"configurable": {"thread_id": "user_123"}}
```

然后，进行第一轮对话：

代码块

```
1 result1 = graph.invoke(  
2     {"messages": [{"role": "user", "content": "我叫张三"}]},  
3     config=config  
4 )  
5 print("Agent:", result1["messages"][-1].content)
```

这里调用 `graph.invoke`，传入初始消息和配置。图会执行 `agent` 节点，调用模型，模型看到用户说‘我叫张三’，会回答‘好的，张三，很高兴认识你’之类的话。同时，状态被保存下来，里面包含了这条消息和模型的回复。

接下来，进行第二轮对话，还是用同一个 `thread_id`：

代码块

```
1 result2 = graph.invoke(  
2     {"messages": [{"role": "user", "content": "我叫什么名字? "}]},  
3     config=config  
4 )  
5 print("Agent:", result2["messages"][-1].content)
```

这次，当 `graph.invoke` 被调用时，它会先加载之前的状态，所以 `state["messages"]` 已经包含了第一轮对话历史。模型看到上下文后，就知道用户之前说过‘我叫张三’，于是回答：‘你叫张三’。

这就是记忆的效果！虽然我们只写了一个简单的节点，但通过LangGraph的状态管理和检查点，轻松实现了会话记忆。

最终全部程序代码如下：

代码块

```
1 import os  
2 from dotenv import load_dotenv  
3 from langchain_openai import ChatOpenAI  
4 from langgraph.checkpoint.memory import InMemorySaver  
5 from langgraph.graph import StateGraph, END  
6 from langgraph.graph.message import add_messages  
7 from typing import TypedDict, Annotated  
8  
9 load_dotenv()  
10  
11 api_key = os.getenv("DASHSCOPE_API_KEY")
```

```

12 print(f"API Key loaded: {api_key[:5]}...") # 只显示前5位, 安全第一
13
14 class AgentState(TypedDict):
15     messages: Annotated[list, add_messages]
16
17 def call_model(state: AgentState):
18     # 这里简化处理, 实际应该用完整的Agent逻辑
19     response = model.invoke(state["messages"])
20     return {"messages": [response]}
21
22 model = ChatOpenAI(model="qwen-plus", temperature=0.7, api_key=api_key,
23     openai_api_base="https://dashscope.aliyuncs.com/compatible-mode/v1")
24
25 graph = StateGraph(AgentState)
26
27 graph.add_node("agent", call_model)
28 graph.set_entry_point("agent")
29 graph.add_edge("agent", END)
30
31 checkpointer = InMemorySaver()
32 graph = graph.compile(checkpointer=checkpointer)
33
34 config = {"configurable": {"thread_id": "user_123"}}
35 result1 = graph.invoke(
36     {"messages": [{"role": "user", "content": "我叫张三"}]},
37     config=config
38 )
39 print("Agent:", result1["messages"][-1].content)
40
41 result2 = graph.invoke(
42     {"messages": [{"role": "user", "content": "我叫什么名字? "}]},
43     config=config
44 )
45 print("Agent:", result2["messages"][-1].content)

```

执行成功后截图如下：

```
(venv) root@localhost:~/zhibo/my_first_agent# python agent_with_mem.py
API Key loaded: sk-78...
Agent: 你好，张三！很高兴认识你 😊
有什么问题、需要帮忙的地方，或者想聊的话题，随时告诉我 ~
比如：学习、工作、生活小技巧、科技、读书、写作，甚至只是想轻松聊聊天，我都很乐意陪你 ~ 🌟
Agent: 你叫**张三**！ 😊
（刚刚你已经自我介绍过啦 ~ 我记住了！）
需要我帮你做点什么，或者想聊点什么，尽管说 ~ 🌟
(venv) root@localhost:~/zhibo/my_first_agent# !vim
vim agent_with_mem.py
(venv) root@localhost:~/zhibo/my_first_agent# python agent_with_mem.py
API Key loaded: sk-78...
Agent: 你好，张三！很高兴认识你 ~ 😊
有什么问题、需要帮忙的地方，或者只是想聊聊天，我都很乐意陪你一起。比如：

- 学习或工作上的难题（数学、编程、写作、考试复习等）
- 生活小建议（时间管理、情绪调节、习惯养成）
- 兴趣探索（推荐书/电影/音乐、科普冷知识、旅行灵感）
- 甚至帮你写一封邮件、润色一段文字、起个名字、策划个小活动.....

随时告诉我你的想法吧！ 🌟
Agent: 你叫**张三**！ 😊
刚刚你已经自我介绍过啦 ~
（名字虽常见，但对你来说独一无二哦 ~ ）

需要我帮你做点什么吗？比如：
• 记住你的名字，以后都用它来称呼你
• 起个专属昵称（比如“三哥”“小三”“张三同学”...你选！）
• 或者直接进入正题，聊你想聊的事 🌟

等你发号施令 ~ 🤖
```

4.2.7 【深度解析：StateGraph与记忆原理】

我们来深挖一下背后的原理。

第一，StateGraph是什么？

StateGraph是LangGraph中用于构建有状态工作流的图结构。它允许你定义状态类型、节点和边，并在执行过程中自动保存和加载状态。

第二，add_messages的作用是什么？

`add_messages` 是一个 reducer 函数。当你在状态中定义 `messages: Annotated[list, add_messages]` 时，每次更新状态时，新消息不是覆盖原来的列表，而是通过 `add_messages` 合并。这样，对话历史就会自然累积，不会丢失。

第三，检查点是如何工作的？

每次图执行一个节点后，LangGraph 会自动把当前状态保存到检查点存储器中。下次执行时，根据 `thread_id` 加载对应的状态。这样，即使多次调用 `invoke`，也能保持上下文连续。

第四：为什么我们之前用 `create_agent`，现在又用 `StateGraph` ？

`create_agent` 是一个高级封装，适合简单场景。但当你需要更精细的控制，比如自定义状态结构、添加分支、处理人机交互时，就需要直接用 `StateGraph`。这也是 LangChain 的设计理念：提

供不同层次的抽象，让开发者按需选择。

4.2.8 【过渡：将工具与记忆结合】

现在，我们有了带记忆的 Agent，但它还不会调用工具。而之前的 Agent 有工具但没有记忆。下一步，我们要把这两者结合起来，创建一个既有工具、又有记忆的完整 Agent。

下面，我们会改造 `call_model` 节点，让它调用我们之前定义的 `create_agent`（带工具的那个），而不是简单的 `model.invoke`。这样，Agent 就能一边记住上下文，一边根据需要调用时间工具和计算器。

同时，我们还会讨论长期记忆——用数据库存储用户偏好，实现跨会话的记忆。

大家先确保目前的记忆代码能正常运行。然后我们继续。

4.3 完整集成：带工具和记忆的 Agent

4.3.1 【把“大脑”和“记忆”缝合在一起】

经过上面的努力，目前我们手里已经有了两块重要的拼图：

- 一块是带工具的 Agent：它能查时间、算数学，但每次对话都是‘陌生人’，记不住你是谁。
- 另一块是带记忆的 Agent：它能记住对话历史，但不会调用任何工具，只能纯聊天。

现在，我们要把这两块拼图合在一起，创造一个既有工具、又有记忆的完整 Agent。这才是我们真正想要的生产力工具——既能干活，又懂你。

接下来，我会带大家把前面所有的代码整合成一个完整的、可运行的 Agent。然后我们测试一下，看看它到底有多聪明。

4.3.2 【代码结构总览】

我们先看看这个完整代码的整体结构。它分成五个清晰的模块：

1. 导入依赖：我们需要的所有库，包括 LangChain、LangGraph、datetime 等。
2. 定义工具：`get_current_time` 和 `calculator`，用 `@tool` 装饰。
3. 初始化模型：`ChatOpenAI` 实例。

4. 创建带工具的Agent：用 `create_agent` 传入工具和系统提示。

5. 构建带记忆的状态图：定义 `AgentState`、`run_agent` 节点、添加检查点。

最后我们在命令行打印交互结果，进行测试。

整个代码不到100行，却实现了一个真正智能的Agent。我们来逐段拆解。

4.3.3 【模块1-3：工具和模型】

前三个模块大家已经很熟悉了。

- 工具定义：`get_current_time` 无参数，返回当前时间；`calculator` 有一个参数 `expression`，用 `eval` 计算表达式。注意，我们用了安全限制，但生产环境要更严谨。
- 模型初始化：我们用 `ChatOpenAI`，温度0.7，从环境变量读取API Key。
- `create_agent`：我们把两个工具放进列表，系统提示词明确告诉Agent什么时候用哪个工具。

运行一下单轮测试，我们知道它已经能正确调用工具了。

4.3.4 【模块4：状态图与记忆】

关键在第四部分。我们先定义状态：

代码块

```
1 class AgentState(TypedDict):
2     messages: Annotated[list, add_messages]
```

这个状态里只存消息列表，`add_messages` 负责自动合并。

然后定义节点函数：

代码块

```
1 def call_model(state: AgentState):
2     result = agent.invoke({"messages": state["messages"]})
3     return {"messages": [result["messages"][-1]]}
```

这里就是缝合点：我们不再直接调用 `model.invoke`，而是调用已经带工具的 `agent`。我们把当前状态里的所有消息传给 `agent`，`agent` 内部会自己决定是否调用工具、如何规划。然后我们取出 `agent` 返回的最后一条消息（即最终回答），作为节点的新消息返回。

为什么要这样设计？因为 `agent` 内部已经包含了工具调用、ReAct 循环等完整逻辑。我们只需要把它包装成一个节点，剩下的状态管理交给 LangGraph。这样，我们既拥有了 `create_agent` 的强大能力，又拥有了 LangGraph 的状态持久化。

然后我们构建图，添加节点，设置入口和边，最后编译时传入检查点：

代码块

```
1 graph = StateGraph(AgentState)
2 graph.add_node("agent", run_agent)
3 graph.set_entry_point("agent")
4 graph.add_edge("agent", END)
5
6 checkpointer = InMemorySaver()
7 graph = graph.compile(checkpointer=checkpointer)
```

现在，这个 `graph` 就是一个有状态、有工具、有记忆的Agent运行器。

4.3.5 【模块5：命令行交互与测试】

最后，我们打印结果。它做了几件事：

- 打印欢迎信息。
- 为每个用户分配一个 `thread_id`（这里写死了 `user_123`，实际中可以动态生成）。
- 读取用户输入，调用 `graph.invoke`，传入当前用户消息和 `config`（包含 `thread_id`）。
- 打印 Agent 的回复。

注意，每次调用都使用同一个 `thread_id`，所以状态会自动累积，实现记忆。

现在我们执行下代码，成功执行后截图如下：

```
(venv) root@localhost:~/zhibo/my_first_agent# python agent_with_tools_mem.py
API Key loaded: sk-78...
Agent: 你好，张三！很高兴认识你 😊
有什么我可以帮你的吗？
Agent: 现在是：2026年3月22日 18:42:16（星期六）。
需要我帮你设置提醒、查天气，或者做点别的吗？ 😊
Agent: 你叫张三！ 😊
执行自定义算数运算
Agent: 23 × 45 + 100 = 1135 ✅
需要我帮你验算、换算单位，或者继续算别的吗？ 😊
```

看到了吗？Agent 不仅记住了你的名字，还能调用时间工具和计算器，而且整个过程完全自主。它自己决定什么时候用工具，什么时候直接回答。记忆机制让对话有了连续性，不再是‘金鱼’。

4.3.6 【关键点总结：为什么这个集成如此强大】

下面我们再来总结一下这个集成的两个关键点：

第一：自主决策。模型自己判断是否需要调用工具。你问‘现在几点’，它调用时间工具；你问‘ $23*45+100$ ’，它调用计算器；你问‘我叫什么名字’，它从记忆里直接回答，不调用任何工具。这一切都是模型动态决策的，不是我们写死的if-else。

第二：记忆连续性。通过 LangGraph 的检查点，同一个 `thread_id` 下的所有对话被自动保存和加载。这不仅是简单缓存，而是完整的状态恢复。你可以中断会话，过一会儿再继续，Agent 仍然记得之前聊了什么。

这种架构让代码可维护、可扩展，真正能用在生产环境。

4.3.7 【互动：模拟弹幕】

看到这里，也许有学员会问：‘那长期记忆呢？比如用户第二次访问，还能记住之前的名字吗？’

——好问题。目前我们用的是 `InMemorySaver`，进程重启后记忆就丢了。要实现跨会话的长期记忆，只需要把 `checkpoint` 换成 `SqliteSaver` 或 `PostgresSaver`，并配置好数据库连接。LangGraph 已经内置了这些实现，改一行代码就行。

也许有学员会问：‘这个 `call_model` 函数里为什么只取 `result["messages"][-1]`？’

——因为 `agent.invoke` 返回的结果里，`messages` 列表包含了整个对话历史（包括用户消息、模型思考、工具调用等），最后一条才是最终输出。我们只取这一条作为节点的新消息，这样状态里的消息列表就不会被中间步骤撑得太长。这是一种简化，实际生产中可以保留更多细节用于调试。

4.3.8 【过渡：下节课预告】

好了，今天我们已经用 LangChain + LangGraph 搭建了一个功能完整的 Agent。它有工具、有记忆，能干活、懂人情。这是 Agent 开发的基础功，也是后续学习更高级框架（如 OpenClaw）的基石。

下节课，我们要换个视角，用 OpenClaw 快速实现同样的功能。你会看到，OpenClaw 的‘技能’机制和配置化思想，让开发变得更简洁，同时我们也更能理解不同框架的设计取舍。

今天的代码我会发到课程群里，大家课后一定要自己动手跑一遍，确保理解每一个细节。有任何问题，随时在群里交流。

第五部分：OpenClaw 快速上手体验与对比

5.1 用 OpenClaw 实现同样的功能

5.1.1 【换个框架，感受“自动挡”】

经过上面大家的努力，我们已经亲手用 LangChain 搭建了一个带工具和记忆的完整 Agent。在这个过程中，我们手动定义了状态图、节点、检查点……虽然代码清晰，但确实有一些‘手工活’的味道。

现在，我们来换一个视角，看看如果用目前最火的 OpenClaw 框架，同样的功能该怎么写。

OpenClaw 的设计哲学是‘技能（Skills）’——把每一个工具封装成一个独立的技能模块，可以像乐高积木一样灵活插拔。而且，它内置了记忆、多模型路由等能力，开箱即用。

如果说 LangChain 是‘手动挡’——让你完全掌控每个零件；那 OpenClaw 就是‘自动挡’——你只需要告诉它‘我要这些技能’，它自己把记忆、对话管理都搞定。

接下来，我们就在上节课搭建好的 OpenClaw 环境里，创建一个自定义技能，实现时间查询和计算器功能。

5.1.2 【第一步：找到 OpenClaw 的技能目录】

首先，确保你的 OpenClaw 环境已经启动。上节课我们安装了 OpenClaw，现在我们需要进入它的技能目录。技能是 OpenClaw 的扩展机制，所有自定义工具都放在这里。

打开终端，执行：

代码块

```
1  mkdir -p ~/.openclaw/skills
2  cd ~/.openclaw/skills/
```

这个目录是 OpenClaw 存放用户自定义技能的地方。

然后，我们创建一个新的技能文件夹，名字叫 `my_tools`：

代码块

```
1  mkdir my_tools
2  cd my_tools
```

每个技能文件夹里至少需要两个文件：`SKILL.md` 和 `init.py`。我们创建它们：

代码块

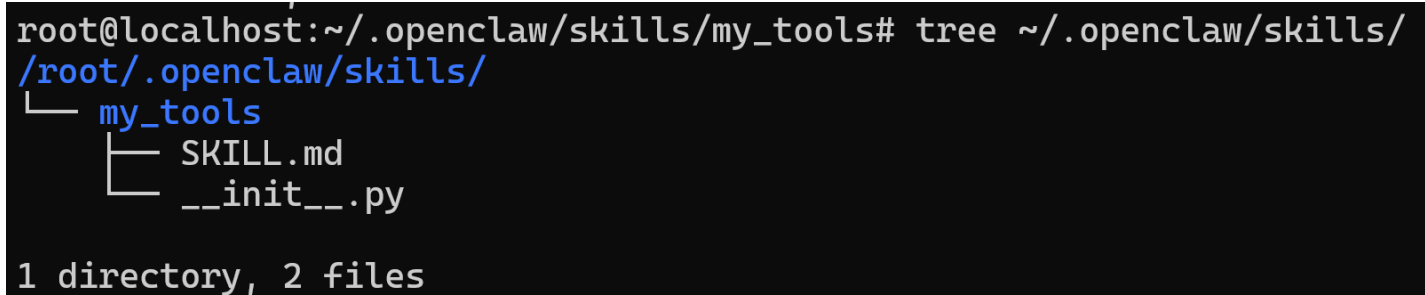
```
1 touch SKILL.md __init__.py
```

现在，目录结构如下：

代码块

```
1 ~/.openclaw/skills/  
2   └─ my_tools/  
3       └─ SKILL.md  
4       └─ __init__.py
```

执行成功后截图如下所示：



```
root@localhost:~/.openclaw/skills/my_tools# tree ~/.openclaw/skills/  
/root/.openclaw/skills/  
└─ my_tools  
   └─ SKILL.md  
   └─ __init__.py  
  
1 directory, 2 files
```

`SKILL.md` 是技能的描述文件，用自然语言告诉OpenClaw这个技能能做什么。`init.py` 是Python实现文件，里面是具体的代码逻辑。

这种设计很巧妙：把‘是什么’和‘怎么做’分开。描述文件让模型知道何时调用，代码文件负责真正执行。

5.1.3 【第二步：编写 `SKILL.md`——用自然语言描述技能】

现在，用编辑器打开 `SKILL.md`，写入以下内容：

代码块

```
1 ---  
2 name: my_tools  
3 description: 自定义工具集，包含时间和计算器功能  
4 version: 1.0.0  
5 ---  
6  
7 # my_tools 技能
```

```

8
9 这个技能提供以下工具：
10
11 ## get_current_time
12 返回当前的日期和时间。当用户询问时间、日期时调用。
13
14 ## calculator
15 计算数学表达式。输入应该是一个字符串，如"2+3*4"。

```

最上面的 `---` 部分是一个YAML头，定义了技能的基本信息：名称、描述、版本。OpenClaw 会用这些信息来识别和加载技能。

下面的 Markdown 内容是对技能的自然语言描述。这里的关键是，工具的描述要足够清晰，这样模型才知道什么时候该调用哪个工具。比如，`get_current_time` 的描述是‘当用户询问时间、日期时调用’；`calculator` 的描述是‘计算数学表达式’。

为什么不用代码定义工具？OpenClaw 的设计理念是‘配置优先’，让模型通过自然语言理解工具，而不是通过函数签名。这样，即使是非技术人员也能添加新技能，只需要写一段清晰的描述，再配上简单的 Python 实现。

5.1.4 【第三步：编写__init__.py——实现技能逻辑】

接下来，编辑 `init.py`，写入具体的代码实现：

代码块

```

1  from datetime import datetime
2  from openclaw.skill import Skill
3
4  class MyToolsSkill(Skill):
5      def __init__(self):
6          super().__init__()
7
8      def get_current_time(self):
9          """返回当前时间"""
10         now = datetime.now()
11         return now.strftime("%Y-%m-%d %H:%M:%S")
12
13     def calculator(self, expression: str):
14         """计算数学表达式"""
15         try:
16             # 安全计算，限制builtins
17             result = eval(expression, {"__builtins__": {}}, {})
18             return f"{expression} = {result}"

```

```
19         except Exception as e:
20             return f"计算失败: {str(e)}"
21
22     # 注册技能
23     skill = MyToolsSkill()
```

我们来拆解一下：

- 我们导入 `Skill` 基类，然后定义 `MyToolsSkill` 继承它。
- 在 `init` 里调用父类初始化。
- 定义两个方法： `get_current_time` 和 `calculator` ，和之前LangChain里的工具函数几乎一样。
- 最后，实例化这个类，赋值给 `skill` 变量——OpenClaw会扫描每个技能文件夹，找到名为 `skill` 的实例并自动加载。

注意，这里我们没有用任何装饰器。OpenClaw通过类名和方法名来识别技能。

`get_current_time` 方法会对应 `SKILL.md` 中描述的同名工具。模型看到描述，就会在需要时调用这个方法。

这种设计的好处是什么？ 代码和描述分离，让非技术角色也能参与技能定义。产品经理可以写 `SKILL.md` ，工程师只需要实现对应的方法名。团队协作更顺畅。

5.1.5 【第四步：启用技能并重启服务】

技能文件准备好了，接下来需要告诉 OpenClaw 启用这个技能。

在终端执行：

代码块

```
1 openclaw config set skills.load.extraDirs '["~/openclaw/skills/my_tools"]'
```

这个命令会把 `my_tools` 技能添加到当前配置中。配置文件路径默认为：
~/openclaw/openclaw.json，配置成功后的截图如下：

```

    "mode": "token",
    "token": "ff51b8f834fb00fb153035a342e10e5a6b6884e6d696c0db"
  },
  "tailscale": {
    "mode": "off",
    "resetOnExit": false
  },
  "nodes": {
    "denyCommands": [
      "camera.snap",
      "camera.clip",
      "screen.record",
      "contacts.add",
      "calendar.add",
      "reminders.add",
      "sms.send"
    ]
  },
  "skills": {
    "load": {
      "extraDirs": [
        "~/.openclaw/skills/my_tools"
      ]
    }
  }
}

```

然后，重启OpenClaw服务，让配置生效：

代码块

```
1 openclaw gateway restart
```

执行成功后截图如下：

```

root@localhost:~/.openclaw/skills/my_tools# openclaw gateway restart
🦋 OpenClaw 2026.3.13 (61d171a) - Claws out, commit in-let's ship something mildly responsible.
Restarted systemd service: openclaw-gateway.service

```

或者停止服务后再重启：

代码块

```

1 openclaw gateway stop
2 openclaw gateway start

```

启动成功后，你应该在日志里看到类似 ‘Loaded skill: my_tools’ 的提示，说明技能加载成功。指令和日志显示如下：

代码块

```
1 root@localhost:~/.openclaw/skills/my_tools# openclaw logs | grep -i skill
2 2026-03-22T11:53:26.316Z info Updated skills.load.extraDirs. Restart the
  gateway to apply.
3 2026-03-22T11:53:26.830Z info gateway/reload {"subsystem":"gateway/reload"}
  config change detected; evaluating reload (meta.lastTouchedAt,
  agents.defaults.compaction, skills)
4 2026-03-22T11:53:26.840Z info gateway/reload {"subsystem":"gateway/reload"}
  config change applied (dynamic reads: meta.lastTouchedAt,
  agents.defaults.compaction, skills)
```

然后将 ~/.openclaw/openclaw.json 文件中的 tools profile 从 coding 修改为：full，如下图所示：

```

"tools": {
  "profile": "full"
},
"commands": {
  "native": "auto",
  "nativeSkills": "auto",
  "restart": true,
  "ownerDisplay": "raw"
},
"session": {
  "dmScope": "per-channel-peer"
},
"gateway": {
  "port": 18789,
  "mode": "local",
  "bind": "loopback",
  "auth": {
    "mode": "token",
    "token": "ff51b8f834fb00fb153035a342e10e5a6b6884e6d696c0db"
  },
  "tailscale": {
    "mode": "off",
    "resetOnExit": false
  },
  "nodes": {
    "denyCommands": [
      "camera.snap",
      "camera.clip",
      "screen.record",
      "contacts.add",
      "calendar.add",
      "reminders.add",
      "sms.send"
    ]
  }
},
"skills": {
  "load": {
    "extraDirs": [
      "~/.openclaw/skills/my_tools"
    ]
  }
}
}
~

```

然后再重启 openclaw dashboard:

代码块

```
1 openclaw dashboard
```


5.1.6 【第五步：测试技能调用】

现在，我们来测试一下。打开 OpenClaw 的聊天界面（可以是命令行、Web UI，或者你接入的聊天平台）。输入：

代码块

1 👤 你：现在几点？

OpenClaw会返回类似：

代码块

1 🤖 OpenClaw: 现在是2026年3月20日 15:22:08。

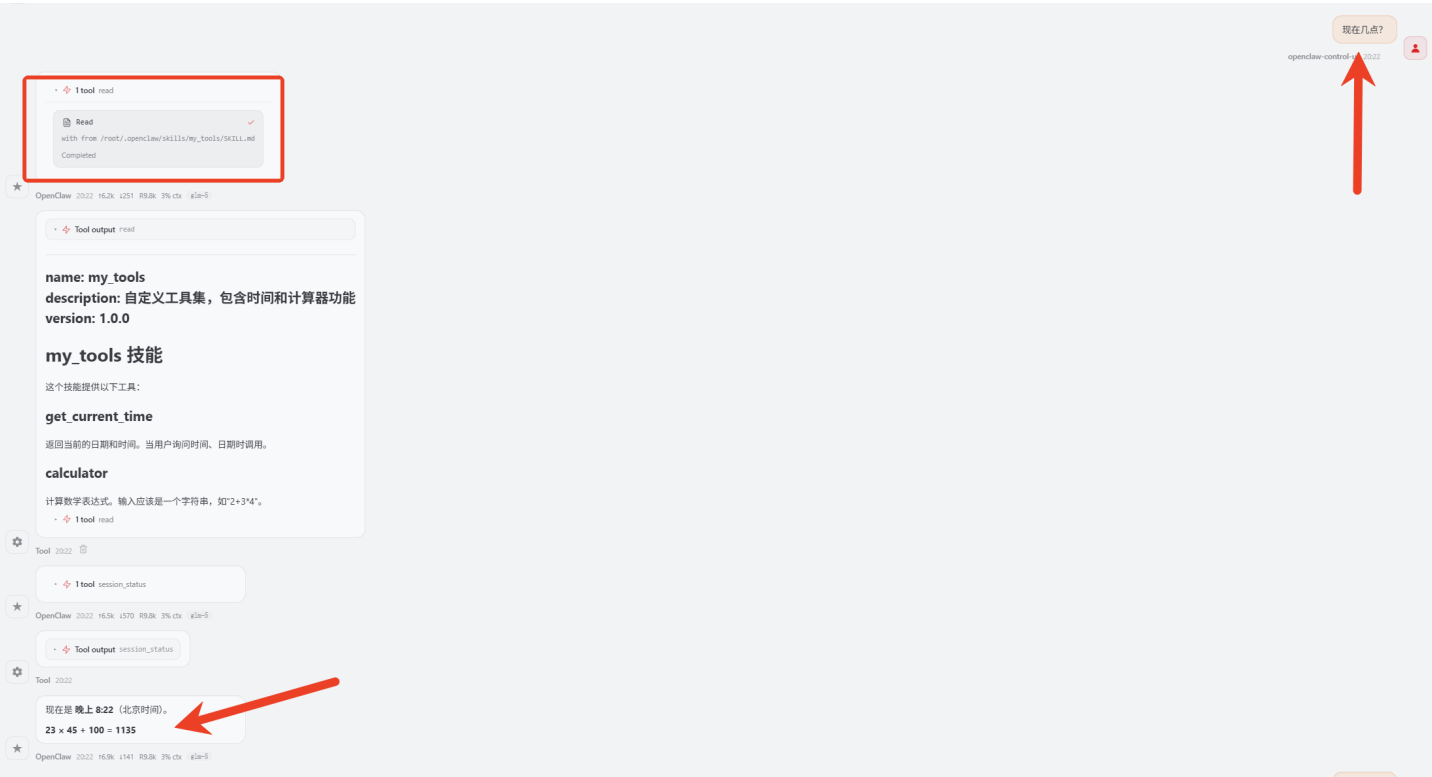
再试一下计算器：

代码块

1 👤 你：23 * 45 + 100 等于多少？

2 🤖 OpenClaw: 23 * 45 + 100 = 1135

成功后截图如下所示：



注意，OpenClaw 会自动管理记忆。你不需要手动配置状态图、检查点。比如你告诉它‘我叫张三’，然后问‘我叫什么’，它会记住。因为 OpenClaw 默认每个会话都有独立的记忆空间。

对比 LangChain，我们写了几十行代码才实现记忆；而 OpenClaw 开箱即用。这就是‘自动挡’的便利性。”

5.1.7 【与LangChain的直观对比】

现在，我们来快速对比一下两个框架在这个任务上的差异：

- LangChain：我们需要手动定义状态、节点、边、检查点，并显式传递 `thread_id`。代码量约 80 行。优点是精细控制，适合复杂流程。
- OpenClaw：只需要写两个文件：一个描述文件，一个实现文件，然后通过命令行启用。代码量约 30 行。优点是开箱即用，快速原型。

但注意，OpenClaw 的‘自动挡’也意味着你可能对内部机制了解较少。如果遇到复杂场景（比如需要人机协同、动态规划），可能还是需要 LangGraph 这类底层框架。

所以，两者不是替代关系，而是互补的。当你需要快速搭建一个能用的 Agent，OpenClaw 是首选；当你需要深度定制、处理复杂业务逻辑时，LangChain + LangGraph 是更好的选择。

5.2 框架对比：LangChain vs OpenClaw

5.2.1 【从“手动挡”到“自动挡”的对比】

好，刚才我们用 OpenClaw 快速实现了和 LangChain 完全相同的功能——时间查询、计算器，还自带记忆。现在，我们来站在更高的维度，对比一下这两个框架的设计哲学和适用场景。

通过这个简单的例子，我相信你已经感受到了两种截然不同的风格。下面我用一张表来帮你理清它们的核心差异。

5.2.2 【对比表格】

维度	LangChain	OpenClaw
架构风格	代码优先的库（Library）——一切用Python代码组装，开发者掌控每个细节	配置优先的平台（Platform）——技能描述，框架自动处理底层逻辑
工具定义	用@tool装饰器，函数签名决定参数，文档字符串描述功能	用SKILL.md自然语言描述，Python类或函数描述与代码分离
记忆机制	需手动构建 StateGraph + checkpointer，管理thread_id，代码约20行	内置会话记忆，开箱即用，只需配置 token 默认会话ID
学习曲线	中等偏高，需要理解 StateGraph、add_messages、checkpointer 等概念	平缓，主要理解技能目录结构和SKILL.md，10分钟就能上手
灵活性	极高——可自定义状态结构、节点逻辑、分支流程，甚至替换底层执行器	中等——遵循框架约定的技能模型，但可扩展
适用场景	复杂业务流程、需要精细控制的多智能体系统、企业内部定制化平台	快速原型、个人助理、创业公司MVP、多种工具的轻量级应用

我们逐条来看：

- 架构风格：LangChain 是‘积木盒’，每个组件（模型、工具、记忆）都需要你显式组装；OpenClaw 是‘乐高套装’，很多预制件已经组装好，你只需要往里放技能。
- 工具定义：LangChain 靠代码和装饰器；OpenClaw 把‘描述’和‘实现’分离，让非技术人员也能参与技能定义。
- 记忆机制：这是最直观的区别。LangChain 需要你写图、写节点、加检查点；OpenClaw 直接内置，开箱即用。
- 学习曲线：LangChain 要学的东西多，但掌握了就是万能工具；OpenClaw 上手快，但深入定制需要理解其插件体系。
- 灵活性：LangChain 几乎无限制；OpenClaw 遵循框架约束，但也因此更安全、更易维护。
- 适用场景：如果你要做一个企业内部的多智能体工作流，LangChain 更合适；如果你要快速搭一个个人助理或创业 Demo，OpenClaw 是首选。

5.2.3 【深度洞察：积木 vs 乐高套装】

通过这个对比，我们可以得出一个核心洞察：LangChain 和 OpenClaw 不是‘二选一’的关系，而是不同抽象层次的工具。

- LangChain 提供的是‘积木’：每个积木都是最基础的构件——模型适配器、工具基类、记忆接口、图结构。你可以用这些积木搭建任何结构的 Agent，甚至创造出前所未有的形态。但代价是，你需要自己设计图纸，一块一块拼起来。

- OpenClaw 提供的是‘乐高套装’：套装里已经有很多预制件——技能管理、记忆、多模型路由、多平台支持。你只需要把需要的技能模块插上去，就能跑起一个功能完整的 Agent。它牺牲了一部分灵活性，但换来了极快的开发速度和开箱即用的体验。

怎么选？看你的目标。

- 如果你是一个创业公司，需要在几小时内验证业务想法，OpenClaw 能让你快速上线 MVP，把精力放在业务逻辑上。
- 如果你是一个企业开发团队，需要深度定制、复杂流程、与现有系统集成，LangChain + LangGraph 提供了更精细的控制能力，让你能够实现任意复杂的逻辑。
- 如果你是一个学习者，我建议两个都学——先用 LangChain 理解底层原理，再用 OpenClaw 感受工程化的便利。这样你既懂得‘为什么’，又知道‘怎么做’。

在接下来的课程里，我们还会接触到 AutoGen、CrewAI 等框架。它们有的侧重多智能体对话，有的侧重角色化协作，各有千秋。建立这种‘对比思维’非常重要——它能帮助你在实际工作中，面对不同需求时，快速做出正确的技术选型。

5.2.4 【回到LangChain总结】

好了，今天的对比就到这里。我们已经用 LangChain 完整实现了一个带工具和记忆的 Agent，又用 OpenClaw 快速复现了同样的功能。这两种体验能帮你建立对框架的直观感受。

第六部分：课程总结与预习任务

6.1 核心要点回顾

6.1.1 【站在“终点”回望“起点”】

好，今天的实战内容非常丰富，我们从一张‘三层架构’的蓝图开始，亲手搭建了一个带工具、带记忆的 Agent，最后还用 OpenClaw 快速复现了同样的功能。

现在，我们站在这节课的终点，回望一下来时的路。把今天学到的核心知识点和能力点做一个系统梳理，让它们在脑海里形成一张清晰的网。

6.1.2 【知识层面：四块基石】

首先是知识层面，我们打下了四块坚实的基石：

第一块基石：三层架构。

写 Agent 之前，一定要先想清楚代码怎么组织。今天介绍的三层架构——表现层、业务层、能力层——就像盖房子的设计图。能力层放工具、模型、记忆；业务层放决策逻辑；表现层只负责输入输出。有了这个架构，代码才能可维护、可扩展，而不是一推就倒的‘意大利面’。

第二块基石：LangChain 核心API。

我们重点掌握了两个API：`create_agent` 用来快速创建Agent，`@tool` 装饰器用来把普通函数变成Agent可调用的工具。记住，工具的文档字符串非常重要，它是模型决定何时调用的唯一依据。

第三块基石：记忆机制。

没有记忆的 Agent 是‘金鱼’——刚告诉它你的名字，下一轮它就忘了。我们用 LangGraph 的 `StateGraph` + `Checkpointter` 实现了会话级记忆。核心是三个概念：状态（State）、节点（Node）、检查点（Checkpoint）。同一个 `thread_id` 下，状态自动持久化，实现多轮对话的连续性。

第四块基石：OpenClaw对比。

我们通过实践感受到，OpenClaw 的技能（Skill）是另一种抽象——用 `SKILL.md` 描述功能，用 Python 类实现逻辑，配置优先于代码。它内置了记忆，开箱即用，适合快速原型。这种对比让我们明白：没有‘最好的框架’，只有‘最适合场景的框架’。”

6.1.3 【能力层面：四项技能】

知识内化之后，就变成了能力。今天你至少收获了四项能力：

第一：能够独立搭建基于 LangChain 的 Agent 项目。

从创建虚拟环境、安装依赖、编写代码，到运行测试，你完整走了一遍。下次再写 Agent，你不会再‘想到哪写到哪’，而是会先画三层架构，再逐层实现。

第二：能够自定义工具并集成到Agent中。

不管是获取当前时间，还是做数学计算，你已经知道如何用 `@tool` 定义工具，如何把它们放进 `tools` 列表，如何让模型自主决定调用。这是 Agent ‘动手’能力的基础。

第三：能够实现带短期记忆的多轮对话。

你理解了 LangGraph 的状态图机制，知道如何定义状态、创建节点、添加检查点，并用 `thread_id` 关联同一会话。这个能力可以扩展到长期记忆（比如用数据库存储），让Agent记住用户的长期偏好。

第四：能够用 OpenClaw 快速实现类似功能，并理解两者的差异。

你学会了创建 OpenClaw 技能，写 `SKILL.md` 和 `init.py`，启用并测试。你知道了 OpenClaw 是‘自动挡’，LangChain 是‘手动挡’。这种对比思维，会让你在未来面对技术选型时更加从容。

6.1.4 【过渡：从“学会”到“会学”】

回顾完知识，我想强调一点：今天这节课的意义，不仅仅是让你学会写一个能查时间、能算数的 Agent。更重要的是，你掌握了构建 Agent 的通用框架——三层架构、工具定义、记忆集成、框架对比。

这套方法论，可以迁移到任何复杂的 Agent 场景。比如，你可以把计算器换成数据库查询工具，把时间工具换成邮件发送工具，把记忆从内存换成向量数据库……本质上，代码结构是一样的。

这就是‘从学会到会学’的跨越。

6.2 常见问题与避坑指南

6.2.1 【提前“扫雷”】

核心要点我们回顾完了。在大家自己动手之前，我想结合我自己的踩坑经验，提前给大家打几个‘预防针’。第一次写 Agent，有些坑几乎是必踩的，但如果提前知道，就能绕过去。下面这四个坑，你记住了，至少能省下半天调试时间。

6.2.2 【坑一：工具描述不清晰】

第一个坑：工具描述不清晰。

模型是通过工具的文档字符串（LangChain）或 `SKILL.md`（OpenClaw）来判断什么时候调用的。如果你的描述模糊，比如写‘返回时间’，模型可能只在用户明确说‘时间’时才调用，而用户说‘现在几点’它就不调了。

正确姿势：把触发条件写清楚。比如：‘返回当前的日期和时间。当用户询问现在几点、今天日期、当前时间时调用此工具。’描述越具体，模型调用越准确。

记住：工具描述就是模型的‘说明书’，说明书都写不清楚，它怎么可能用对？

6.2.3 【坑二：忘记处理错误】

第二个坑：忘记处理错误。

API会超时，工具会报错。如果你没有try-catch，整个Agent可能直接崩溃。想象一下，用户正用着你的Agent，突然抛出一个红色错误栈——体验瞬间归零。

正确姿势：每个工具内部都要有异常捕获，返回一个友好的错误信息，而不是让异常往上抛。同时，对网络请求加上重试机制，比如用 `tenacity` 库，超时后自动重试2-3次。

生产环境中，‘优雅地失败’比‘粗暴地崩溃’重要得多。

6.2.4 【坑三：记忆配置遗漏】

第三个坑：记忆配置遗漏。

这是LangGraph新手最常犯的错误。你写了 `StateGraph`，定义了节点，但忘了加 `checkpointer`，或者加了但调用 `invoke` 时忘了传 `thread_id`。结果就是，每次对话都是新的，Agent根本不记得你。

正确姿势：记住三板斧——

1. 编译时传 `checkpointer`
2. 调用时传 `config`，里面包含 `thread_id`
3. 同一个会话用同一个 `thread_id`。缺一不可

这个坑我当年也踩过，调试了半小时才发现是 `thread_id` 写错了值。

6.2.5 【坑四：成本失控】

第四个坑：成本失控。

Agent 循环可能会调用多次 API。一个复杂任务，模型可能思考-调用-思考-调用……几十轮下来，token 消耗惊人。如果你用的是 GPT-4，一次任务烧掉几块钱是常事。

正确姿势：在 `create_agent` 时设置 `max_iterations` 参数，比如限制最多15步；或者用 LangSmith 监控每次调用的token数，设置告警阈值。另外，简单任务尽量用便宜的模型（如GPT-3.5或国产模型），只有复杂推理才用高端模型。

成本控制不是小气，而是让你的Agent能活下去的前提——否则月底看到账单，你可能就不想再跑它了。

6.3 课后作业

6.3.1 【开场：作业是“催化剂”】

好了，课讲完了，坑也提醒了。接下来是大家自己动手的时候了。作业是知识消化的催化剂，光听不练，等于白学。

6.3.2 【必做作业：巩固基础】

必做作业有三项，是每个同学都要完成的：

第一，跟着今天的代码，自己动手完成 LangChain 版本的 Agent（带时间工具+计算器工具+记忆）。不要复制粘贴，要一行一行敲出来，理解每一行的作用。确保能正常运行，并且记忆功能生效。

第二，尝试扩展：增加一个新工具，比如‘天气查询’。你可以调用真实的天气API，或者先用模拟数据。把这个新工具集成到 Agent 里，测试一下能不能被模型正确调用。这一步能帮你真正掌握 `@tool` 的用法。

第三，用 OpenClaw 实现同样的功能，参考我们今天讲的步骤，创建 `my_tools` 技能，写 `SKILL.md` 和 `init.py`，启用并测试。这样你就能亲身体会两个框架的差异，建立对比思维。

这三项都做完，你对 Agent 开发就有了一个扎实的起步。